



TrafficStar: Connection-Type Fingerprinting of Tor Users

Reducing Anonymity by Detecting Connection Types

Anslutningstyp Fingeravtryck av Tor-användare

Reducera Anonymitet Genom Dektering av Anslutningstyper

Kristoffer Norman <Kristoffer.Norman@proton.me>

Faculty of Health, Science and Technology

Master thesis in Computer Science

Second Cycle, 30 hp (ECTS)

Supervisor: Tobias Pulls, Karlstad University <tobias.pulls@kau.se>

Examiner: Matthias Beckerle, Karlstad University <matthias.beckerle@kau.se>

Karlstad, June 19th, 2026

Abstract

As time passes, connection types (network technologies) continue to be developed or deployed. Two of these technologies are 5G and Starlink. As the number of used network technologies grows and disperses, some technologies will become less popular and others more popular. For example, Starlink had over 9 million active customers in the year 2025. Tor and VPN services rely on the fact that users hide among other users of the service by blending in, so that you can't identify them with enough characteristics that separate them from the users of that service. In this thesis, we use a repurposed deep-learning-based Website Fingerprinting attack, Tik-Tok, to detect the connection type a client uses. We do this *from the vantage point of the server* by analyzing the inter-arrival times of packets at the server in a classical client-server setting when the user connects through the Tor anonymity network or a VPN service. The connection types we consider are Ethernet, WiFi, 5G, and Starlink. We collected datasets for Tik-Tok with a self-developed tool—TrafficStar—that automatically creates VPN and Tor connections, and generates and records network traces of *upload traffic* between a client and a server. The resulting datasets are used to evaluate Tik-Tok as a *connection-type fingerprinting* attack.

The results show that when a user uses Tor, the recall of detecting 5G was 84.9% with a precision 86.0%. Starlink had a recall value of 82.5% and a precision of 72.7%. The models for Tor were unable to distinguish between Ethernet and WiFi, something the rest of the models could do. For multi-hop VPN connections (2 relays), recall was 99.4% and precision 97.0% for 5G, whilst Starlink had recall of 90.6% and precision 89.0%. When no relays were used, the average accuracy and recall were 99.7%. Unfortunately, the number of samples collected for one-hop VPN connections was deemed too small, as the models performed worse than those for multi-hop VPN connections, with a sample size of only 25% compared to multi-hop connections. We conclude that while connection-type fingerprinting with Tik-Tok is not perfect, information about users' connection types is detectable to some extent by servers, even over Tor.

Keywords

Connection-type fingerprinting, Network Access Technologies, Tor, VPN, AI.

Sammanfattning

Allt eftersom tiden går utvecklas och etableras nya anslutningstyper (nätverkstekniker). Två av dessa tekniker är 5G och Starlink. Allt eftersom antalet använda nätverkstekniker växer och sprids kommer vissa tekniker att bli mindre populära och andra mer populära. Till exempel hade Starlink över 9 miljoner aktiva kunder 2025. Anonymitetsnätverket Tor och VPN-tjänster är beroende av att användare gömmer sig bland andra användare genom att smälta in. Med andra ord ska en attackerare inte kunna särskilja användare utifrån olika egenskaper, såsom operativsystem, webbläsare eller anslutningstyp. I denna avhandling använder vi en omarbetad djupinlärningsbaserad webbplatsfingeravtrycksattack, Tik-Tok, för att upptäcka vilken anslutningstyp en klient använder. Vi gör detta *från serverns perspektiv* genom att analysera ankomsttiderna för paket till servern i en klassisk klient-server-miljö när användaren ansluter sig via Tor eller en VPN-tjänst. De anslutningstyper vi överväger är Ethernet, WiFi, 5G och Starlink. Vi samlade in dataset för Tik-Tok med ett egenutvecklat verktyg—TrafficStar—som automatiskt skapar VPN- och Tor-anslutningar samt genererar och spelar in nätverksspår av upload-trafik mellan en klient och en server. De resulterande dataseten används för att utvärdera Tik-Tok som en attack för att identifiera anslutningstyper.

Våra resultat visar att när en användare använder Tor är träffsäkerheten för att upptäcka 5G 84,9% med en precision på 86,0%. Starlink hade en noggrannhet på 82,5% och en precision på 72,7%. Modellerna för Tor kunde inte skilja mellan Ethernet och WiFi, något som resten av modellerna kunde göra. För multi-hop VPN-anslutningar (två reläer) var träffsäkerheten 99,4% och precisionen 97,0% för 5G, medan Starlink hade träffsäkerheten 90,6% och precisionen 89,0%. När inga reläer användes var den genomsnittliga noggrannheten 99,7%. Tyvärr visade sig antalet insamlade prover för one-hop VPN-anslutningar vara för litet, eftersom modellerna presterade sämre än multi-hop VPN-anslutningar, med en urvalsstorlek på endast 25% jämfört med multi-hop-anslutningar. Vi drar slutsatsen att även om fingeravtryck av anslutningstyp med Tik-Tok inte är perfekta, kan information om användarnas anslutningstyper i viss utsträckning detekteras av servrar, även över Tor.

Nyckelord

Anslutningstyp fingeravtryck, nätverksteknik, Tor, VPN, AI.

Acknowledgements

I would like to personally write down my gratitude to Tobias Pulls for being a kind and guiding hand throughout this thesis.

It's only right that the authors of the paper *A Detailed Characterization of Starlink One-way Delay*—Johan Garcia, Simon Sundberg, and Anna Brunström—are mentioned as well, for providing analysis that was a part of how Tobias Pulls came up with the thesis topic.

Matthias Beckerle has my gratitude as well for taking the time and effort to grade this thesis and provide feedback.

Acronyms

API	Application Programming Interface
CLI	Command Line Interface
CPU	Central Processing Unit
CTFP	Connection-Type Fingerprinting
DF	Deep Fingerprinting
DL	Deep Learning
DNN	Deep Neural Network
HTTP	Hyper Text Transfer Protocol
ISP	Internet Service Provider
ML	Machine Learning
NIC	Network Interface Card
RAM	Random Access Memory
REST	Representational State Transfer
VM	Virtual Machine
VNIC	Virtual Network Interface Card
VPN	Virtual Private Network
WF	Website Fingerprinting

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Description	2
1.3	Thesis Objective	2
1.4	Thesis Goals	2
1.5	Ethics and Sustainability	3
1.6	Methodology	3
1.7	Delimitations	3
1.8	Outline	4
2	Background and Related Work	5
2.1	Tor – Onion routing	5
2.2	OnionMasq	6
2.3	VPNs and Wireguard	6
2.4	Connection Types and Inter-Arrival Time	6
2.4.1	Transmission Collisions and Medium Access Control	7
2.4.2	Ethernet and CSMA/CD – IEEE standard 802.3	7
2.4.3	WiFi and CSMA/CA – IEEE standard 802.11	7
2.4.4	5G – Fifth Generation Mobile Communications	8
2.4.5	Starlink	8
2.5	tcpdump	8
2.6	Connection Type and Website Fingerprinting	8
2.6.1	Deep Learning and Models	9
2.6.2	Deep Neural Network (DNN) Model Metrics	9
2.6.3	Dataset Splitting and K-Fold Cross-Validation	10
2.6.4	Deep Fingerprinting	10
2.6.5	Tik-Tok Website Fingerprinting	11
2.7	Related Work	11
3	Methodology	12
3.1	Environment and Threat Model	12
3.2	Traffic Generation and Recording – TrafficStar	13
3.3	Analysis – Connection Type Fingerprinting	14
4	Design and Implementation	16
4.1	TrafficStar	16

4.1.1	Client and Test Handlers	16
4.1.2	VPN Implementation	17
4.1.3	Tor Implementation	19
4.1.4	Server	19
4.2	Parser	20
4.3	Tik-Tok Executor	21
5	Data Collection	22
6	Results and Discussion	26
6.1	Tor	26
6.2	Virtual Private Network (VPN) — Multi-hop	29
6.3	VPN — One-hop	34
6.4	No Relays	36
6.5	Summary	38
7	Conclusions and Future Work	40
	References	42

Chapter 1

Introduction

Every day, over two million people use the anonymization network *Tor* for internet communication [15]. *Tor* is the most deployed anonymization network in the world and continues to grow. Among the users of *Tor* are individuals who are trying to circumvent censorship, protect their privacy, journalists, and governments [37].

Tor provides its users with anonymity (to a degree) by obscuring the origin of a user's internet traffic, which is achieved by redirecting the user's internet traffic through three *Tor* nodes. The fact that a person is using *Tor* is not anonymous by default, since, for example, an Internet Service Provider (ISP) could see that the user is communicating with the *Tor* network [37]. This means a user is only anonymous among other *Tor* users, and that anonymity protection is affected by the number of users [9, 29].

Tor is not the only anonymity tool used today that is growing. The VPN network industry is growing and has become a \$30 billion industry in the year 2020 [8]. Just as with *Tor*, amongst its users are people using it for censorship-circumvention and privacy. The internet traffic of a VPN user is redirected through 1 or 2 VPN servers to provide anonymity [8, 41, 70].

1.1 Background

The anonymity provided by *Tor* or VPNs, however, is susceptible to techniques that deanonymize users or their activities [22, 29]. One of these techniques is Website Fingerprinting (WF). WF inspects the network packets between a user and the anonymity network to detect the websites a user is visiting [36], and one of these attacks, Tik-Tok, has achieved accuracies up to 98% by using Deep Learning (DL) [46].

Meanwhile, as the *Tor* network grows and VPN adoption increases, new connection types are emerging, such as 5G and Starlink. These new connection types are filling gaps in current types and allowing more people to gain high-speed internet access [30, 35]. These two newly specified connection types exhibit distinct inter-arrival patterns

of network traffic compared to Ethernet and WiFi [4, 11, 21, 47].

1.2 Problem Description

In this thesis, we aim to study the degree to which these new connection types affect Tor and VPN users' inter-arrival patterns of network traffic, and to what extent a recipient server could distinguish the connection types a user uses by inspecting the inter-arrival patterns of a Tor or VPN user. The connection types of interest are: Ethernet, WiFi, 5G, and Starlink. The number of relays¹ used will include 1 (one-hop VPN connection), 2 (multi-hop VPN connection), 3 (Tor circuits), and none. We call the process of inspecting inter-arrival times to detect the connection type a user uses *Connection-Type Fingerprinting (CTFP)*. The research question of this thesis is: “*To what extent are users' network access technologies detectable from the vantage point of destination servers based on packet inter-arrival times, despite users using multi-hop relays?*”

1.3 Thesis Objective

The objective of the thesis is to present results from network measurements using four network access technologies, evaluating CTFP attacks using Tik-Tok on the resulting datasets. The thesis also briefly presents TrafficStar, the tool designed for data collection, only to the degree necessary to understand how the datasets were collected.

1.4 Thesis Goals

The goals of the degree project are to:

- Implement a tool (TrafficStar) that automatically creates VPN and Tor connections, uses the connections to upload data from a client to a server, and has the server record the network traffic.
- Collect datasets to evaluate the use of Tik-Tok for CTFP.
- Determine if network access technologies create sufficiently rich differences in network traffic that adversaries, from the vantage point of destination servers, can with some probability distinguish between different users' network access technologies.

¹Relay refers to, in this thesis, a device that the user's internet traffic will be routed through before being forwarded to the internet, e.g., A Tor node

1.5 Ethics and Sustainability

The ethical issues that relate to private individuals are, in our view, minimal, as we only record network traffic related to our measurements, and the server never sends data to connections on an address being recorded. Tor has its own safety guidelines that specify what to take into consideration when performing research related to Tor [61].

One of the safety guidelines states that one should *use a simulated Tor network when possible*, which is not feasible for this thesis, as we are interested in whether the patterns of 5G and Starlink survive the noise of the real Tor network. The load we put on the network is low as well, since when we use Tor, bandwidth is limited to 1 Mbps.

Another guideline states that *“the benefits should outweigh the risks”*. If the thesis successfully detects connection types, further research should study the implications of these findings. If the findings are serious, this thesis may be an initial step in designing defenses against related attacks.

Other guidelines exist, e.g., *“It’s safest to only attack yourself / your own traffic”* and *“Consider whether the user meant for that data to be private”*. As noted above, we do not actively collect data from or fingerprint private users.

1.6 Methodology

As the information in Section 1.4 suggests, an empirical methodology is used, focused on experimental internet measurements [1]. A client will use different connection types (Ethernet, WiFi, 5G, and Starlink) to upload data to a recipient via a VPN or Tor connection. The recipient will record the network traffic and store it in a dataset, along with the connection type used. The data is then to be used for training and validating repurposed Tik-Tok models with 10-fold cross-validation.

1.7 Delimitations

In this thesis, we only use one VPN provider. Further, we only use one network configuration for each connection type; all of the network configurations are a part of the Computer Science research laboratory network at Karlstad University, and we only use the four connection types available there: Ethernet, WiFi, 5G, and Starlink. Similarly, we use Virtual Machines (VMs) instead of physical machines and use only two instances of VMs. Finally, we evaluate only the scenario where a client uploads data to the server (adversary).

1.8 Outline

The thesis is structured as follows. In Chapter 2, we describe the necessary background and related work. In Chapter 3, we describe the methodology, and in Chapter 4, the implementation details. In Chapter 5, we discuss the data collection process. In Chapter 6, we present and discuss the thesis results, and in Chapter 7, we conclude the thesis.

Chapter 2

Background and Related Work

In this chapter, we present the background and related work for the thesis.

2.1 Tor – Onion routing

When a user is using onion routing for protecting their privacy, their network packets to the internet traverse multiple nodes called *onion relays* before reaching their final destinations [3]. Each relay is responsible for forwarding the received packets to the next relay or to its final destination. The path the packet takes for a connection is called a *circuit*. The relays can *only* obtain information required for forwarding the packets to the next relay or destination. This is achieved through layered encryption, with the entry guard being the only relay aware of the user's IP address, whilst the exit relay is the only relay aware of the destination IP address [3, 55].

In the case of Tor, the circuit commonly consists of three relays. The first relay (the one that the user directly communicates with) is called the *entry guard*, the second relay is called the *middle relay*, and the final relay is called the *exit relay* [29]. Since the relays only have information needed for forwarding the packet to the next relay or destination, the only way to de-anonymize the user is by performing traffic analysis, or if the relays conspire with each other [3, 29].

Currently, it is believed that only very capable adversaries can reliably determine which end-host a Tor user is communicating with, and vice versa, since existing methods rely on observing significant fractions of the entire internet for performing traffic analysis, or inserting malicious onion relays that conspire with each other [3, 29]. Since the Tor user is in charge of selecting the onion relays and there are strict constraints of circuit generation to mitigate collaboration, this method is currently only feasible for a very capable adversary since the Tor network consists of over 9000 relays [54, 62, 63].

2.2 OnionMasq

OnionMasq is a tool developed by the Tor project for creating Virtual Network Interface Cards (VNICs) that proxy internet connections over Tor circuits [60]. OnionMasq creates a *TUN*¹ interface. All network packets sent through the OnionMasq's VNIC are intercepted and routed over a Tor circuit. For creating and managing Tor circuits, OnionMasq uses the Rust crate *Arti*, an implementation of the Tor protocol [44]. *Arti* is modular and allows programs to create their own implementations of TCP connections to the Guard relay by providing *Arti* with a custom *NetStreamProvider* [44]. *Arti* uses a *state directory* to store information used by the Tor protocol to mitigate relay collaboration.

2.3 VPNs and Wireguard

VPN stands for Virtual Private Network, and its use cases largely vary from accessing office resources to protecting privacy and avoiding censorship [8, 68].

When a user uses a VPN for all of their internet traffic, their internet traffic will be redirected to the VPN provider through a tunnel over the internet, and the VPN provider acts as a relay between the user and the rest of the internet [70]. Common VPN protocols like WireGuard only specify how tunnels, authentication, and related processes between users and VPN providers are performed, leaving it up to the VPN provider to implement the relaying aspect [7].

In the normal case, only one relay is used in VPN connections, but some VPN providers allow 2 relays, for example, Mullvad [41]. We call VPN connections using one relay *one-hop* VPN connections, and those using two relays *multi-hop* VPN connections.

Since the traffic is being tunneled to the VPN provider over the internet, a connection type like Ethernet or WiFi must be used in combination with the VPN by users to connect to the VPN. The effects on the inter-arrival patterns of IP packets may vary significantly depending on the VPN provider used, the processing applied to the packets, and other factors.

2.4 Connection Types and Inter-Arrival Time

Different connection types (e.g., WiFi, Ethernet, or Starlink) have different effects on the inter-arrival patterns of IP packets because the processes affecting them vary. For example, when using an Ethernet connection to a network, the algorithm allowing multiple nodes to utilize a shared physical medium for transmission is the MAC (Medium Access Control) algorithm CSMA/CD [6], whilst in the case of WiFi, it is controlled by the algorithm CSMA/CA [69].

¹A TUN interface is a VNIC in Linux that allows programs to handle network packets [32].

In the following sections, we describe the four considered connection types and theories of how the inter-arrival time of packets can be affected by them, but also the challenges that might occur and their solutions.

2.4.1 Transmission Collisions and Medium Access Control

When two or more nodes are using the same physical medium, the physical medium is referred to as a shared physical medium. Depending on the solution, multiple transmissions on a shared physical medium can occur at once without problems, or not. WiFi does not work with simultaneous transmission and therefore uses a MAC protocol [5, 17, 18]. There are different MAC protocols, and their characteristics vary, but they all attempt to control the access to the shared medium to prevent collisions, and if collisions occur, they try to recover [69].

2.4.2 Ethernet and CSMA/CD – IEEE standard 802.3

Ethernet is a wired network protocol, meaning devices communicating with Ethernet uses physical cables for communication. Modern Ethernet solutions are commonly connected through switches and full duplex connections², preventing transmission collisions from ever occurring [51]. Ethernet uses CSMA/CD as its MAC scheme [17]. CSMA/CD primarily uses techniques called carrier detection and interference detection for detecting and preventing transmission collisions [34]. The inter-arrival pattern of IP packets is highly consistent if full-duplex connections are used and a transmitter only has to wait for the IPG (Interpacket gap). IPG refers to a time gap that must exist between transmissions according to the Ethernet standard, and allows participating network nodes to prepare for the next frame being transmitted. The size of it varies depending on the Ethernet variant used [17].

2.4.3 WiFi and CSMA/CA – IEEE standard 802.11

WiFi is a wireless network technology that implements IEEE 802.11 protocols, and devices using WiFi communicates with a network through radio transmissions. The physical medium used for data transmission is a shared one, so transmission collisions are possible [18].

WiFi has several reliability issues. If two transmitters start transmitting at the same time, they will keep transmitting until they are finished, which might produce indecipherable results at receiving nodes [42]. Another significant problem is background interference from other devices that produce radio signals. Other significant problems are the hidden terminal problem and aggressive rate control.

²Full duplex connections refer to a connection type where a connected node can send and receive data at the same time.

The reliability issues of WiFi, combined with its shared medium, *may* significantly affect the inter-arrival pattern of IP packets, as packets may be lost or delayed.

2.4.4 5G – Fifth Generation Mobile Communications

5G is the fifth generation of mobile communications and is a cellular network technology. This means that all devices using 5G use radio transmissions to connect to cellular networks [30, 57].

5G uses a scheduling algorithm based on time slots, meaning that devices will be given a slot in time to transmit data. This is distinct from Ethernet and produces inter-arrival time patterns that differ from Ethernet [23, 47].

2.4.5 Starlink

Starlink is a network technology that allows users to access the internet by communicating with satellites [11, 35].

Starlink has over 9 million customers as of 2025 [52], and its network characteristics are currently being studied [35]. Starlink is a closed system, meaning its implementation and technology are not publicly available. However, a recent study, “*A Detailed Characterization of Starlink One-way Delay*” [11], has found that Starlink has unique inter-arrival time patterns in its uplink. This work inspired the research question of this thesis.

2.5 tcpdump

To analyze the effects of different connection types on inter-arrival times, network traffic needs to be recorded. To record the actual IP packets arriving at and leaving a user or server, tools such as tcpdump can be used—a lightweight program for capturing network traffic. tcpdump can save captured traffic to files and filter what traffic to record based on IP addresses [13, 58].

2.6 Connection Type and Website Fingerprinting

To determine the connection type(s) a user uses based on the inter-arrival times of IP packets, some form of analysis is required. It is commonly called WF (Website Fingerprinting) when encrypted traffic is used to fingerprint the websites users visit [31]. The common attack pattern used in WF is that an adversary passively listens to the traffic coming to and from a user. The user in WF typically uses some form of relay, and the adversary listens to traffic between the user and the relay, trying to determine websites visited based only on patterns in encrypted network traffic [16, 19, 31].

In this thesis, we instead assume an adversary that has access to the network traffic at the vantage point between the relay and the destination server, passively records it, and aims to determine the connection type used by the user. We refer to this as *CTFP* (*Connection type fingerprinting*) — to stay in spirit with naming the style of WF.

The following Sections 2.6.1–2.6.5 describe relevant concepts and techniques for fingerprinting the connection type by packet inter-arrival times in this thesis, but also the testing techniques used for the classification method.

2.6.1 Deep Learning and Models

DNNs are the backbone of many modern artificial intelligence solutions existing today, such as language translation and cancer detection [56]. Training a DNN falls under a field referred to as DL, and training a DNN does not require hand-crafted features like Machine Learning (ML) and can use raw data instead [25, 46].

There are different types of DNN models, and their structures usually vary depending on the task; for example, U-Net is a structure used for image segmentation [27]. A model consists of weights and biases (parameters), which are tuned during training. The models are trained in "iterations", and each iteration is referred to as an epoch. When these models are used to predict membership or to make decisions, this process is referred to as inference [25, 27, 56].

2.6.2 DNN Model Metrics

When evaluating a DNN model, two common metrics used are *recall* and *precision*. To explain these metrics, the confusion matrix seen in Figure 2.6.1 is used. The confusion matrix is for a hypothetical DNN model that is supposed to classify whether a sample is a dog, cat, fish, or bird. The *actual* axis states what results were for samples of a class. For example, the second row shows what the model predicted the class labels were for the cat samples. The *predicted* axis states which class the model predicted for each sample.

The *recall* of a class for a DNN model is the number of correctly identified samples of that class, divided by the total number of samples in that class. For the example in Figure 2.6.1, the recall for cats is $\frac{3148}{262+3148+112+138} \approx 0.86 \approx 86\%$.

The *precision* for a class for a DNN model is the rate at which the predictions that something belonged to the class were correct. For the example in Figure 2.6.1, the precision for cats is $\frac{3148}{100+3148+206+410} \approx 0.81 \approx 81\%$.

Another common metric is *accuracy*, which is the proportion of predictions that are correct. For the example in Figure 2.6.1, the accuracy of the model is $\frac{3534+3148+2878+2534}{\text{total amount of predictions}} = \frac{3534+3148+2878+2534}{14640} \approx 0.83 \approx 83\%$.

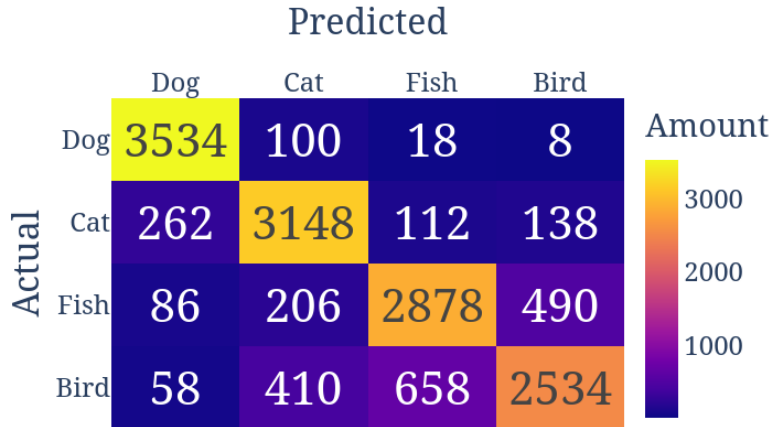


Figure 2.6.1: An example of a confusion matrix for a DNN model that separates between dogs, cats, fish, and birds

2.6.3 Dataset Splitting and K-Fold Cross-Validation

A model needs to be trained and validated. To perform these tasks, the data is split into different sets. A training, validation, and test set. In this thesis, we only split the data into a training and a test set, because the implementation of the Tik-Tok attack does not pick the final model based on the validation dataset; rather, it uses the model from the last epoch. The training set is used to train a model’s parameters, and the test set is used to evaluate the model [25].

To improve robustness and gain a clearer understanding of how the model performs, K-fold cross-validation can be used [67]. In K-fold cross-validation, the training data is partitioned into K folds, and the model is trained on K-1 folds and then validated on the remaining fold. K different splits are evaluated, each using a different fold as validation, and the average metric is used as the K-fold cross-validated metric.

2.6.4 Deep Fingerprinting

Deep Fingerprinting (DF) was introduced in the paper “*Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning*” [50]. DF is a DL approach for WF, achieving accuracies of up to 98% in closed-world³ settings. DF employs convolutional neural networks (CNNs), a type of deep neural network (DNN), to classify network traffic traces. As input features, DF uses only the packet direction [50].

³Closed world refers, for example, to when not all possible websites are considered, but only a subset.

2.6.5 Tik-Tok Website Fingerprinting

Tik-Tok is an extension of DF that is highly relevant to this thesis because, in addition to packet directions, Tik-Tok considers the timestamps of packets as input features. It was presented in the paper “*Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks*” and in closed-world settings, it achieved accuracies similar to DF (98%) [46, 50].

2.7 Related Work

There are works that conduct studies similar to this thesis; however, as far as we know, none have attempted to detect the connection type of users while they are using an anonymization network.

In one paper, “*Classification of access network types: Ethernet, wireless LAN, ADSL, cable modem or dialup?*” [66], Wei et al. performed CTFP by using inter-arrival times, but did not involve any anonymization network or relays. The paper aims to enable a server to detect a client’s connection type by cooperating with the client. Even if the paper did not include any relay, its results indicate that Ethernet connections could be separated from wireless connections using inter-arrival times.

As discussed in Section 2.6, WF is used to detect the website a user visits. The paper, “*WFCAT: Augmenting website fingerprinting with channel-wise attention on timing features*” [12], used a hand-crafted feature they called *Inter-Arrival Time Histograms* to perform WF and achieved accuracies up to 94.47% on detecting websites in closed-world settings when Tor was being used. As the name suggests, the feature relates to inter-arrival times. The paper discussed in Section 2.6.5, Tik-Tok [46], used timestamps as well for WF.

Both WFCAT and Tik-Tok show that timing features can be used even when Tor is being used; however, they only used the timing features for WF, not CTFP. Wei et al. [66] showed that connection types could be detected using inter-arrival times of packets, but never attempted to detect the connection type when a user was behind Tor or an VPN connection. This thesis closes the gap in the aforementioned paper by using WF methods to detect the connection type a user uses when Tor or VPNs are in use.

Chapter 3

Methodology

This chapter describes how the thesis research question is answered. First, Section 3.1 describes the environment for data collection and the assumed threat model. Section 3.2 explains *how data is collected* for answering the research question, and Section 3.3 explains how the research question *is answered* using the data collected.

3.1 Environment and Threat Model

As the project involves fingerprinting the connection types of users behind a Tor or multi-hop VPN connection, we pick a client–server setting for data collection. Figure 3.1.1 shows the network structure used for data collection in this thesis. The data-collection scenario and threat model are that a client uploads a large amount of data to the server, mimicking a “file upload”, and that the server uses the arrival times of client packets to determine which connection types the client is using. The server is therefore the adversary. We assume that the server has no other information at its disposal, such as knowledge of connection-type base rates (e.g., fewer users use Starlink than Ethernet) or insights into congestion control algorithms. As in Website Fingerprinting (Section 2.6.4), we assume that the server is a passive adversary.

The client and server are two VMs provided by Karlstad University, and their different network interfaces are described in Table 3.1.1. Both of the VMs run Ubuntu with the Linux Kernel Version 6.8.0-111-generic and are configured to run the TCP congestion algorithm Cubic. The specifications can be seen in Specification 3.1.1.

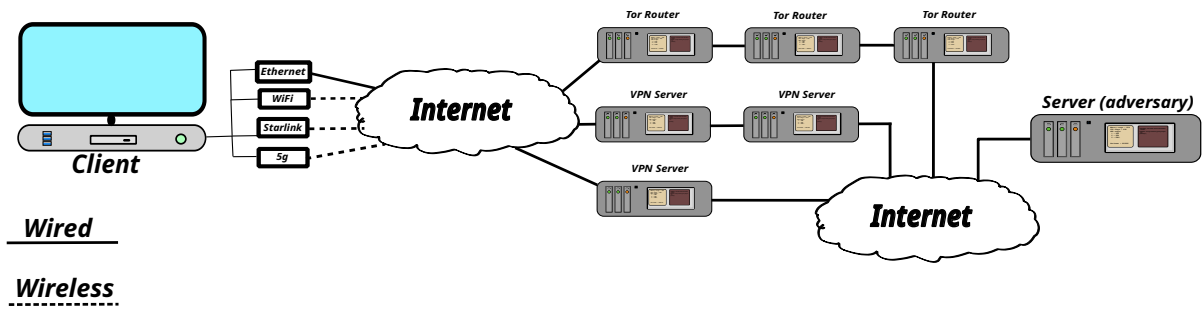


Figure 3.1.1: Overview of the network structure for the Data Collection. Solid lines illustrate wired connections, and dashed lines wireless connections

Specification 3.1.1: VM Specifications	
Operating system	
Distribution: Ubuntu 24.04.3 LTS	
Kernel: Linux KVM - 6.8.0-111-generic	
TCP Congestion Algorithm: Cubic	
Hardware	
Central Processing Unit (CPU): Intel Xeon Silver 4214R	
Random Access Memory (RAM): 16GiB	

Table 3.1.1: The link types and bandwidths for the different internet connections used by the client and server in the research project

Client		Server	
Link Type	Bandwidth (to server)	Link Type	Bandwidth (to client)
Ethernet	1 Gbps	Ethernet	1 Gbps
WiFi	100 Mbps		
Starlink	≈10 Mbps		
5G	≈100 Mbps		

3.2 Traffic Generation and Recording – TrafficStar

To answer whether a server can determine the connection types of a client it is serving based on the inter-arrival times of packets, a dataset needs to be generated. To collect and generate network traffic data, a program called TrafficStar was developed. TrafficStar automatically generates network traffic between a client using specific connection types and a server that records it.

On the client VM, TrafficStar starts by reading configuration files that specify which network interfaces exist and their types (e.g., WiFi, Ethernet). The configuration files

also state what kind of data gathering should be performed, which interface to use for the data collection, and, potentially, that Tor or VPNs should be used in combination with the interface.

To generate network traffic for the dataset, the TrafficStar program on the client will first set up a Tor or VPN connection if needed and then connect to a TrafficStar instance running on the server. The client sends the server information about the data to be collected, and the server records the information, creates a socket, and starts a tcpdump instance listening to the address of the socket, which records the network packets arriving and leaving the server along with their arrival or departure time. This kind of recording is referred to as a *network trace* in this thesis.

The server will then wait for any ongoing data collection to finish and then send the client the address of the socket. The client then proceeds by connecting to the received address and starts dumping traffic for a duration specified in the configuration files. Meanwhile, the server reads all the data sent from the client and discards it until the socket closes. This type of data transfer is referred to as a sink. Once the socket is closed, the tcpdump instance is stopped, and the data gathered from tcpdump is saved into a data structure that preserves the information received from the client.

The client will repeat the data generation process a number of times specified by the configuration files, and each iteration becomes a sample of the dataset.

To prevent information leakage through different bandwidths of the network configurations, TrafficStar on the client is executed in containers that have their *bandwidth limited to 1 Mbps* by routing all network traffic related to the containers into a VNICs. These VNICs have their bandwidth throttled by utilizing the traffic controller built into Linux.

An overview of how a data collection looks for Tor with Ethernet can be seen in Figure 3.2.1.

3.3 Analysis – Connection Type Fingerprinting

To perform CTFP on the collected data, a repurposed WF attack named Tik-Tok is used. Tik-Tok is built upon the idea of DF, using DL for fingerprinting. The DNN models for Tik-Tok use two features of packets from network traces for WF: timestamps and direction (arrival or departure).

To use the Tik-Tok model for CTFP, the data collected from TrafficStar has to be transformed into the format that the Tik-Tok attack uses. Therefore, the data is processed, filtered, and cleaned into the appropriate format. However, unlike the normal case for Tik-Tok, the formatted data only contains the packets originating from the client, since the thesis is related to fingerprinting on the traffic from the client. Also, the timestamps are converted into the inter-arrival times of these packets. The formatted data is used to train several Tik-Tok models for CTFP and are tested using

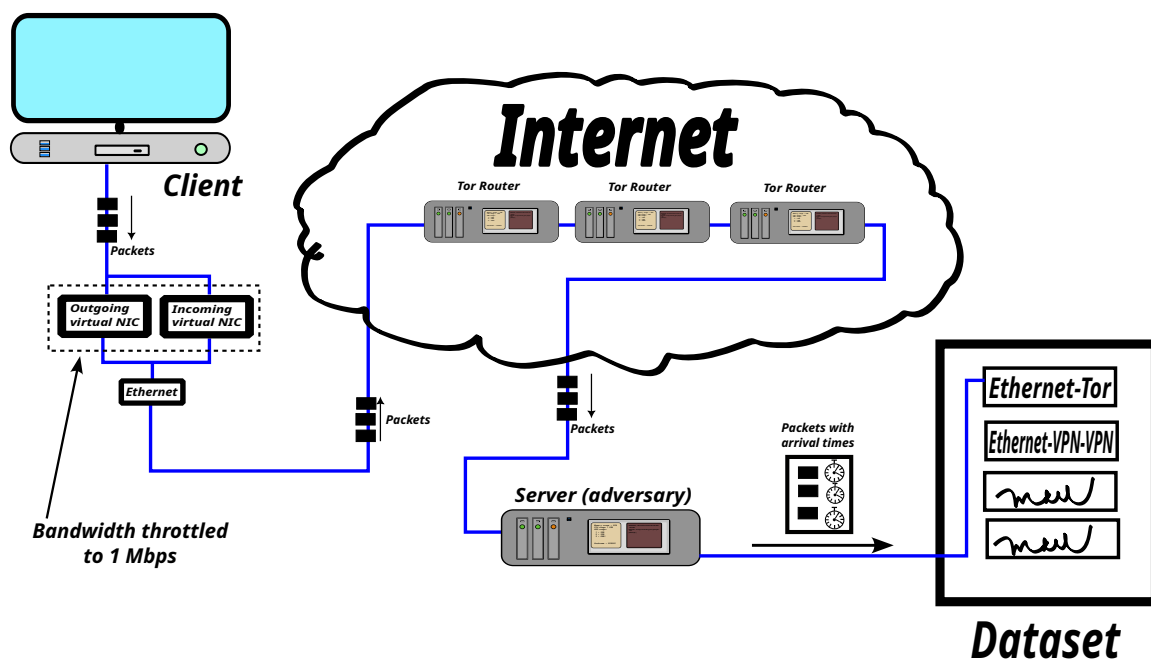


Figure 3.2.1: Overview example of how data collection occurs with Ethernet being relayed over a Tor circuit

10-fold cross-validation.

Chapter 4

Design and Implementation

The following sections describe the design and implementation of the systems developed for answering the research question. The first section, Section 4.1, describes the program used for data collection, TrafficStar. Section 4.2 describes the software used for formatting the collected data from TrafficStar into the format expected by the Tik-Tok model. Appendix A contains key code snippets.

4.1 TrafficStar

TrafficStar, as mentioned in Chapter 3, is the program used for data collection. The program was developed specifically for this thesis and is developed in Rust.

The program consists of two logical parts: client and server logic. The program is launched via a Command Line Interface (CLI) and gets fed a set of arguments that it uses, e.g., to decide whether the client or server logic is to be executed. The client logic is described in Section 4.1.1, and the server in Section 4.1.4.

4.1.1 Client and Test Handlers

The code relevant to the client is the most extensive one in terms of scopes. For example, unlike the server, the client has to have the ability to automatically create VPN and Tor connections. How VPN connections are established is described in Section 4.1.2, and Tor connections in Section 4.1.3.

The client starts by reading several files that describe the tests to run and the available interfaces. Listing 4.1 shows a file describing an interface, and Listing 4.2 a test configuration file. With the configuration files, TrafficStar creates instances of *Test Handlers*. In total, there are three types of test handlers: Mullvad, Tor, and the *Pure* handler. The Mullvad handler handles data collection with VPN connections, the Tor handler handles Tor circuits, and the Pure handler handles connections without a proxy or tunnel. TrafficStar provides the test handlers with a test configuration during their initialization, and the handlers derive test samples from it.


```
1 {
2     "interface_name" : "wifi0",
3     "ipv4" : "172.17.0.2",
4     "ipv4_public" : "81.231.235.221",
5     "tag" : "Wifi",
6     "fwmark" : 10,
7     "data_traversals" : ["Internal","Wifi"]
8 }
```

Listing 4.1: A file used by TrafficStar describing an interface

```
1 {
2     "sinkparams" : {
3         "time" : 60
4     },
5     "test_parameters" : {
6         "name" : "wifi_test_sink",
7         "samples" : 1000,
8         "parallel" : 1
9     },
10    "do_for_tag" : ["Wifi","TorW","OnehopW","TwohopW"]
11 }
```

Listing 4.2: A file used by TrafficStar describing a test to run

TrafficStar tells the initialized test handlers, one by one, to run a sample, continuing until each sample has finished. The test handlers create a VNIC for the sample, if necessary, and establish a connection to the server. The client tells the server the connection types that are being used and requests the address of a *Sink*. If the connection types contain VPNs, the client additionally informs the server of the VPN servers used. A sink in TrafficStar is where data is sent to and read from, but the receiver immediately discards the retrieved data. Once the client receives the address, the client connects to the Sink with the appropriate interface and starts sending a large amount of data into the Sink, attempting to use the full capacity of the connection. Once the Sink has run for a duration specified in the configuration file, the client closes the Sink and informs the server that the test sample has finished.

4.1.2 VPN Implementation

For the VPN implementation, the VPN provider *Mullvad* is used. The Mullvad VPN client application code is available as open-source [40], meaning it could be inspected. Studying the open-source code revealed that Mullvad exposes a Representational State Transfer (REST) Application Programming Interface (API) for the management of what they call *devices*. Documentation for the API also exists [38] and makes it clear that the devices contain variables relevant for WireGuard interfaces. Additionally, the API lists the different VPN servers available from Mullvad and various pieces of information needed for creating WireGuard interfaces that use the listed servers. For example, it contains the IPv4 address of the VPN servers. However, the information also contains a number named *multihop_port* [39]. The name of it indicates that it

is used for multi-hop VPN connections. This port number is unique and only belongs to one of the listed VPN servers. Investigation reveals that using one of the unique port numbers instead of the default WireGuard port (51820) when connecting to any of the listed VPN servers results in WireGuard traffic being routed to the holder of that number, creating a multi-hop VPN connection. The automatic creation of VPN connections is implemented and is described below.

When TrafficStar wants to create a new VPN connection, it starts by generating a new WireGuard key pair (public and private key) by using a crate called *wireguard_keys*. With the keys generated, an Hyper Text Transfer Protocol (HTTP) POST request requesting a new Mullvad device to be created alongside the public key is sent to the API. The API, in turn, responds with the information of the newly created Mullvad device. With the API response, TrafficStar has all the necessary information to create WireGuard interfaces that use the VPN servers listed in the API. An example response is seen in Listing 4.3 and parts of the relevant code in Listing A.1.

```

1 {
2   "id": "e44fb82c-7840-435d-80a7-cd841f38e745",
3   "name": "wise prawn",
4   "pubkey": "9Dw/kgNSCFj9gWumUwc8pHJJj6PcbA1ijTpDlhS8EwI=",
5   "hijack_dns": false,
6   "created": "2026-05-12T10:01:30+00:00",
7   "ipv4_address": "10.136.237.163/32",
8   "ipv6_address": "fc00:bbbb:bbbb:bb01:d:0:8:eda3/128",
9   "ports": []
10 }
```

Listing 4.3: Mullvads API response when creating a new Mullvad device

TrafficStar creates a pool of Mullvad devices in a separate thread so it can create WireGuard interfaces more quickly, and when needed, it extracts a device from the pool. TrafficStar deletes the device when it is no longer in use, and the separate thread creates a new device and adds it to the pool.

When TrafficStar has all the information required to create a VPN connection available, TrafficStar creates a WireGuard interface by using a Rust crate called *wgctrl*. TrafficStar starts by reserving a name for the interface, preventing any other ongoing part of the application from using the same name. With the reserved name, TrafficStar uses *wgctrl* to perform the system calls necessary for setting up the interface and uses specific *firewall marks* to ensure that the VPN traffic uses the correct Network Interface Card (NIC). Code for the creation of the WireGuard interfaces can be seen in Listing A.2.

When running the client, the test handler for VPN connections uses configuration files describing what VPN servers and interfaces to use. The configuration files contain regular expressions that are matched to the VPN servers listed by the API. Listing 4.4 shows a configuration file and Listing A.3 parts of the VPN test handler.

```

1 {
```

```
2  "entry" : "(se-got-wg-003|es-mad-wg-102|no-osl-wg-102|us-phx-wg-208|fr-  
    bod-wg-002|ca-mtr-wg-304|ie-dub-wg-103|jp-osa-wg-004|ar-bue-wg-002|bg-  
    sof-wg-001|be-bru-wg-101)",  
3  "exit" : null,  
4  "tag" : "OnehopW",  
5  "do_for_tags" : ["Wifi"]  
6 }
```

Listing 4.4: A file used by TrafficStar for creating VPN connections

4.1.3 Tor Implementation

For creating Tor connections, TrafficStar uses the program *OnionMasq* from the Tor project (see Section 2.2). However, TrafficStar uses a slightly modified version of *OnionMasq*. First, the modifications allow *OnionMasq* to use a specific NIC for Tor connections; second, they resolve a bug in the original version that limits the bandwidth; and third, they make the *state directory* location configurable.

When starting *OnionMasq*, TrafficStar provides the program with the interface name to use when creating the TCP connection to the Guard relay, achieved by the modifications made to a *NetStreamProvider* in *OnionMasq*, as seen in Listing A.4.

In the original version, *OnionMasq* has a maximum TCP sender window size¹ of 4096 bytes, which prevents full bandwidth utilization. The modified version increases this to 512 Kilobytes, allowing for full bandwidth utilization.

TrafficStar provides its modified version of *OnionMasq* with a newly created state directory. This modification prevents *OnionMasq* from maintaining state and simulates the behavior of a first-time *OnionMasq* Tor connection.

When running the client, the test handler for Tor uses configuration files that specify the interfaces to use for creating Tor connections. TrafficStar creates a VNIC for *OnionMasq*, assigns it an IP address, a name, a firewall mark, and a temporary directory as the state directory. Listing A.5 shows part of the code from the Tor test handler.

4.1.4 Server

The server has the simplest logic because it does not need to establish Tor or VPN connections. The server starts by reading the configuration files describing the NICs on the machine and then waits for clients to connect to one of the specified interfaces. An argument provided to TrafficStar at launch determines which interface to listen to.

¹The TCP sender window size is the maximum amount of bytes of network packets that can reside in the Kernel memory of a sender before the receiver acknowledges that they have received a packet.

Once a client connects to the server, TrafficStar creates a new thread dedicated to handling the client. The client will tell the server it wants to collect data of a certain connection type and request a *Sink* address. If the client is collecting a VPN sample, the client will also send the VPN servers used. The thread will enter a queue for running a Sink, preventing concurrent recordings of data samples. When it finishes waiting in the queue, it will create a new Sink and a *tcpdump* instance listening on the Sink's address. Thereafter, it will send the Sink's address to the client. The server will then wait for the client to signal that it has finished before closing the Sink and saving the network trace from *tcpdump*, along with the information retrieved from the client, into a directory with a structure based on the connection types received from the client.

4.2 Parser

TrafficStar outputs raw network traces that cannot be directly used by the Tik-Tok attacks. Therefore, a parser written in Python cleans, filters, and formats the raw network traces into a format that can be used by the Tik-Tok models.

The parser starts by reading arguments specifying where the network traces to be processed are located, and where to place the parsed data. The parser then uses plugins that take network traces as input and produce filtered, processed network traces as output. Every network trace from the output gets written to a file in a specific format. The files contain rows of data, each containing the following details for a packet: Timestamp, arriving or departing packet, and packet size.

The parser uses two plugins that take the raw network traces and filter out any packets from the server. Both plugins calculate the inter-arrival times of the remaining packets, use them as timestamps for the output, and remove the first and last 250 packets to prevent the Tik-Tok model from fingerprinting based on TCP startup or shutdown characteristics, focusing only on steady-state file upload in our selected scenario.

One of the plugins processes network traces into segments of 1000 packets and produces one output file per segment, but prevents related segments from being in the training or testing set simultaneously. The other plugin keeps 4000 packets, produces only one segment per trace, and discards the rest of the processed network trace.

The plugin creating segments of 1000 packets is used for fingerprinting on VPN connections, and the latter for the others. The reason for segmenting VPN traces is that the number of traces collected for them is small compared to Tor, and through experimentation, we found that obtaining a larger training sample size through segmentation yields better results for VPN connections. The reason for not using fewer packets per segment is that the current segment sizes yield a training set size comparable to Tor's, and during experimentation, using fewer than 1000 packets per segment (for example, 250) did not substantially degrade or improve results.

4.3 Tik-Tok Executor

To train and execute Tik-Tok models, a modified script implementing Tik-Tok from the associated Github repository of the paper “*Maybenot: A Framework for Traffic Analysis Defenses*” is used [45]. A separate script, referred to as the *Executor* in this thesis, will take the parsed data from the Parser and execute it with the script from Maybenot.

The original version of the script did not randomize which samples became a part of the testing or training set (due to particular dataset structures), whereas the modified version does. Another change is that the original version converts timestamps to seconds and rounds the timestamps to the nearest 100 microseconds, whereas the modified version keeps the original timestamps to preserve fine-grained timing information.

The modified version of the script also considers VPN servers when splitting the dataset into training and testing sets. It prevents a VPN server from being in the training and testing set simultaneously. Splitting based on servers makes the assessment more generalizable, as the models are being assessed on servers they have not seen before.

The modified version of the script will also ensure that the number of samples per connection type within a set is the same for each connection type by removing samples from connection types (known as undersampling). Having an equal number of samples per type helps with accuracy. Balanced classes make performance assessments less difficult as well, since one does not have to account for one class having more samples, making accuracies equal to their recall values [26].

Chapter 5

Data Collection

For data collection, each client instance of TrafficStar runs in a *container*¹. The container grants stronger guarantees that clients, for example, do not accidentally use the wrong interface for VPN or Tor connections.

Each container has one VNIC that's exposed to the internet, and the VNIC uses one of the NICs on the client for the internet connections. An example of the commands used to start a container using Starlink can be seen in Listing 5.1.

```
1 docker network create \  
2     --driver bridge \  
3     --opt com.docker.network.bridge.enable_ip_masquerade=true \  
4     --opt com.docker.network.container_iface_prefix=starlink \  
5     --subnet=172.18.90.0/24 \  
6     --opt com.docker.network.bridge.name=br-starlinknet starlinknet && \  
7 ip route add default dev starlink-uplink table 900 && \  
8 ip rule add from 172.18.90.0/24 table 900 && \  
9 \  
10 sysctl -w net.ipv4.conf.all.rp_filter=0 && \  
11 sysctl -w net.ipv4.conf.default.rp_filter=0 && \  
12 docker run -dit --name starlink \  
13     --cap-add=NET_ADMIN \  
14     --cap-add=NET_RAW \  
15     --cap-add=CAP_SYS_ADMIN \  
16     --privileged \  
17     --network=starlinknet \  
18     --sysctl net.ipv4.conf.all.rp_filter=0 \  
19     --sysctl net.ipv4.conf.default.rp_filter=0 \  
20     --device /dev/net/tun \  
21     -e TRAFFICSTAR_MULLVAD_ACCOUNT=$TRAFFICSTAR_MULLVAD_ACCOUNT_STARLINK \  
22     trafficstar
```

Listing 5.1: Commands used to start a container using the Starlink interface present on the client VM

¹When a program is executed in a container, Linux will only expose specified resources to the container [28].

To prevent Tik-Tok models from fingerprinting on the bandwidth, the *egress* bandwidth is *throttled* to 1 Mbps by using the built-in module *police* in Linux's Traffic Controller. The module *police* starts dropping packets to maintain a specified transmission rate if the bandwidth exceeds the rate [20]. Ingress traffic from the internet traverses another VNIC before reaching the container and uses the *police* module to limit the bandwidth to 1 Mbps. The commands used to set up the bandwidth limit can be seen in Listing 5.2.

```

1 ip link add ifb0 type ifb
2 ip link set ifb0 up
3 tc qdisc del dev br-starlinknet root
4 tc qdisc del dev br-starlinknet clsact
5 tc qdisc del dev br-starlinknet ingress
6 tc qdisc del dev br-starlinknet handle ffff: ingress
7 tc qdisc add dev br-starlinknet clsact
8 tc filter add dev br-starlinknet egress protocol ip prio 1 u32 match ip src
   0.0.0.0/0 police rate 1mbit burst 128kb drop
9 tc filter add dev br-starlinknet ingress protocol ip u32 match u32 0 0
   action mirred egress redirect dev ifb0
10 tc qdisc add dev ifb0 clsact
11 tc filter add dev ifb0 egress protocol ip prio 1 u32 match ip src 0.0.0.0/0
   police rate 1mbit burst 128kb drop

```

Listing 5.2: Commands to limit the bandwidth of a container

The server that the clients connect to is also running in a container, but has no bandwidth limitations or anything of that nature. The server, however, had disabled several offloading features of its NIC that affect the timestamps of network traces [10, 33, 59]. These offloading features reduce the amount of work the CPU has to perform for network traffic. The features disabled were *TCP segmentation offloading*, *large receive offloading*, and *generic receive offloading*.

The client runs their sinks for approximately *60 seconds*, and the server prohibits concurrent recordings of network traces. The clients can, however, connect to the server whilst a recording is in progress. Each recording of a connection type, e.g., WiFi, took approximately two days.

Table 5.0.1 shows the number of samples that were used for training and validating Tik-Tok models when Tor was used.

Table 5.0.1: Samples used for Tor.

Connection type	Samples
Ethernet	548
5G	548
Starlink	548
WiFi	548

Table 5.0.2 shows the number of samples used for training and validating models for multi-hop VPN models. A total of 11 VPN servers were used, all of which are listed in Table B.0.1 in Appendix B. Note that some samples didn't appear in either the training or validation set for certain fold numbers, since the Tik-Tok executor prevents a VPN server from being present in the validation and training set at the same time.

Table 5.0.2: Samples used for multi-hop models.

Connection type	Samples training	Samples validation	Unique VPN servers used
Ethernet	576	16	11
5G	576	16	11
Starlink	576	16	11
WiFi	576	16	11

Table 5.0.3 shows the number of traces that contain a specific VPN server for the *one-hop* VPN dataset. A total of 11 VPN servers were used, all of which are listed in Table B.0.2 in Appendix B. Note that the servers used are the same as those in the multi-hop dataset. The data collection window was shorter than expected due to unexpected behavior in Onionmasq; therefore, we placed higher priority on Tor and multi-hop VPN connections over one-hop VPN connections, since we expected the difficulty of CTFP to be more difficult when more relays were used and that more data would be needed for the models to learn the underlying patterns that distinguish the connection types. The number of samples per class in a validation fold is the same for all classes, and the number of validation samples per class varies from 13 to 16 for one-hop VPN, depending on the fold.

Table 5.0.3: Samples used for one-hop models.

Connection type	Samples training	Samples validation	Unique VPN servers used
Ethernet	130	13-16	11
5G	130	13-16	11
Starlink	130	13-16	11
WiFi	130	13-16	11

Table 5.0.4 shows the number of samples that were used for training and validating Tik-Tok models when no relays were used.

Table 5.0.4: Samples used for CTFP when no relays are used.

Connection type	Samples
Ethernet	560
5G	560
Starlink	560
WiFi	560

Chapter 6

Results and Discussion

The data collected from the experiment were parsed using the parsers described in Section 4.2 into a format that could be used to train Tik-Tok attacks. The parsed data was used to train several Tik-Tok models, and 10-fold cross-validation was used to evaluate the models. In the following sections, we describe the results of our analysis. The following sections are ordered by the number of relays used (Tor, multi-hop, one-hop, no relays), and each section describes how their related Tik-Tok models performed. The number of relays used should increase the difficulty of CTFP, as the noise from the internet and different relays intensifies.

6.1 Tor

The average 10-fold accuracy as a function of training epochs can be seen in Figure 6.1.1. As seen in the figure, the average accuracy stops improving significantly around 30 epochs and starts fluctuating between 55-66.6% with a maximum accuracy of 66.6% at 125 epochs, indicating that further training will not yield significant improvements. The training accuracies of the folds continue to rise towards 100% after 30 epochs and reach it before 150 epochs (not seen in the figure), but it *does not* significantly decrease the average test accuracy, which could occur when a model overfits¹ to the training data. However, this does not seem to occur. One possible explanation is that Tik-Tok models are trained with techniques to mitigate overfitting [46, 53]; another possibility is that the models overfit to the training data but are still able to maintain their generalizability², known as *Benign Overfitting* [2, 14].

The dataset for CTFP with Tor is considerably smaller than the one used in the original Tik-Tok paper (2192 vs 85500 samples) [46], potentially meaning that higher accuracies could be achieved with more samples. However, the reported accuracy could be overestimated as well, since k-fold cross-validation can overestimate the

¹Overfitting refers to when a model remembers each sample in the training set instead of the patterns present in the set.

²A model is said to generalize if it can make accurate decisions on unseen data

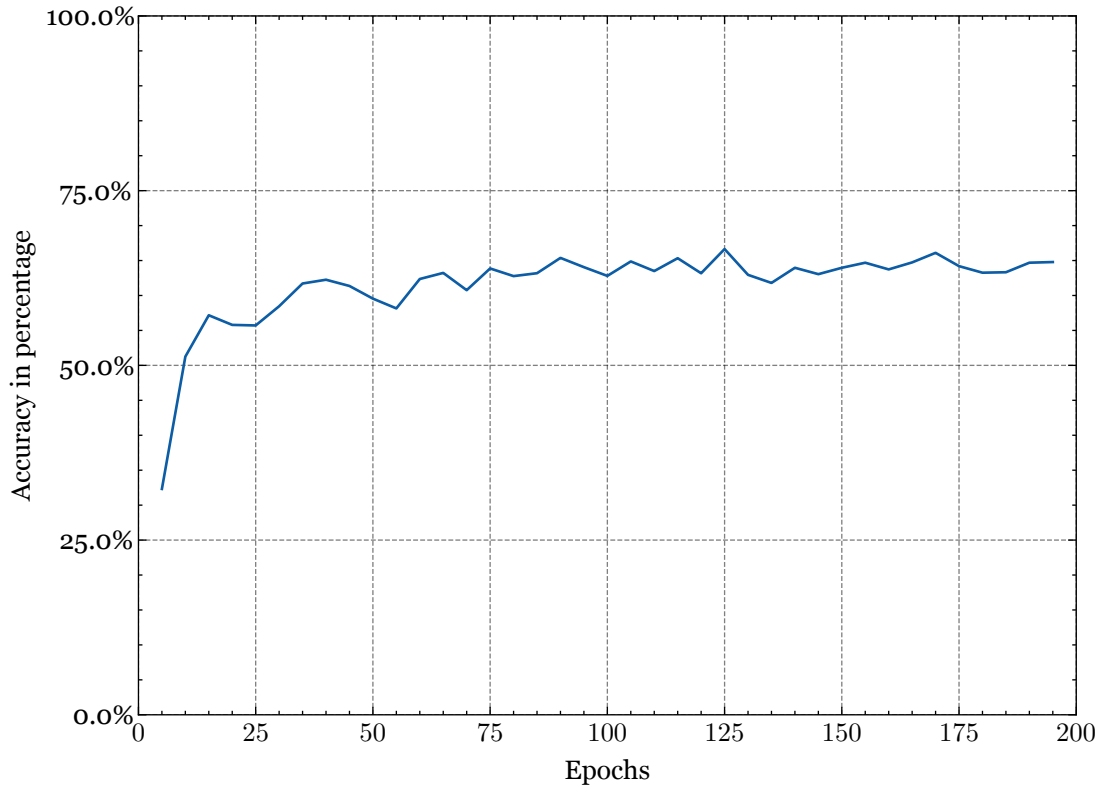


Figure 6.1.1: Average 10-fold cross-validation accuracy for Tor for the different Epochs.

accuracy when using a small number of samples [49, 65]. As seen in Figure 6.1.2, the performance of different folds varies and could also be an artifact of the number of samples.

As seen in Table 6.1.1, the models trained for 125 epochs were, on average, most successful at detecting 5G connections with an average recall value of 84.9%. The models were also decent at detecting Starlink connections, with a recall of 82.5%. The models, however, were less successful on Ethernet and WiFi, achieving recall values of 53.8% and 45.4%, respectively. The difference between precision and recall was relatively small for 5G and Ethernet compared to Starlink and WiFi.

Table 6.1.1: Average CTFP recall of all folds for connection types behind a Tor circuit, 125 Epochs.

Connection type(s)	Recall (%)	Precision (%)
Ethernet	53.8	54.1
WiFi	45.4	51.4
5G	84.9	86.0
Starlink	82.5	72.7
Ethernet or WiFi (Same models)	83.4	88.8
5G or Starlink (Same models)	89.5	84.4

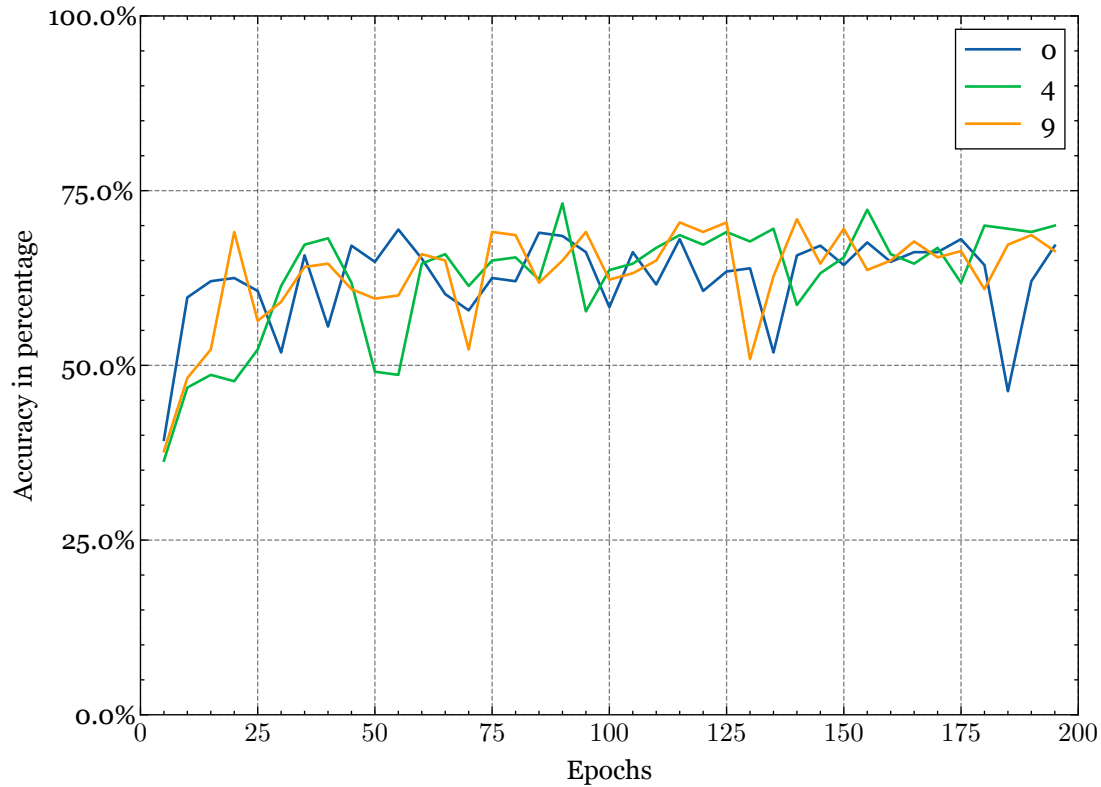


Figure 6.1.2: The fold performance of folds 0, 4, and 9 for Tor for the different Epochs.

The sum of the confusion matrices from the Tor models can be seen in Figure 6.1.3 for 125 Epochs. As seen in the confusion matrix, the model was able to separate between the connection types 5G and Starlink, but not between Ethernet and WiFi. A possible explanation why it could not differentiate between Ethernet and WiFi is that the inter-arrival characteristics of these technologies are not strong enough to be detectable at the server after traversing Tor circuits.

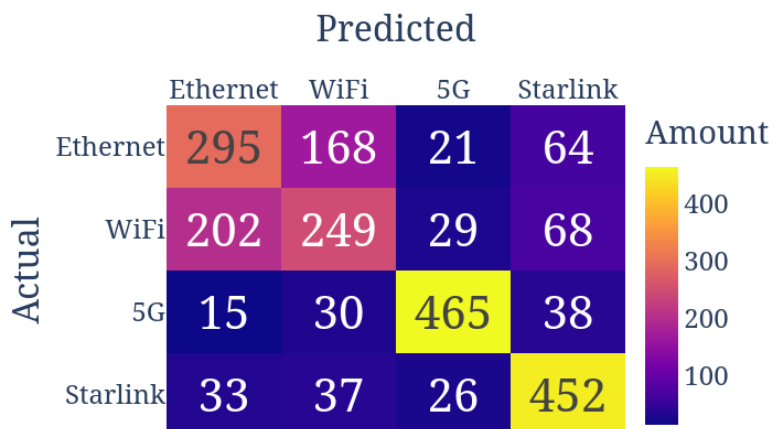


Figure 6.1.3: Sum of the confusion matrices from the Tor models, 125 Epochs.

Another hypothesis is that the higher recall values for 5G and Starlink are partly

produced by the different *latencies* of the connection types *to the Guard relays*; however, as noted in the paper “*Distributed Performance Measurement and Usability Assessment of the Tor Anonymization Network*” [37], the latency of Tor circuits varies significantly, making it unlikely that the only signal the models used for CTFP was the latency. If the models picked up on the latency of the connection types, this could also be an interesting finding, since latency correlates with geographical distance and Tor relays are not evenly distributed geographically [24, 37].

If we group together the classes Ethernet and WiFi from Figure 6.1.3 as one group, and the remaining two as a second group, we get a new confusion matrix as seen in Figure 6.1.4. From the figure, we can see that the models misclassified a 5G or Starlink connection type as WiFi or Ethernet 10.5% of the time, whilst the reverse was 16.6%, a difference of 6.1%.

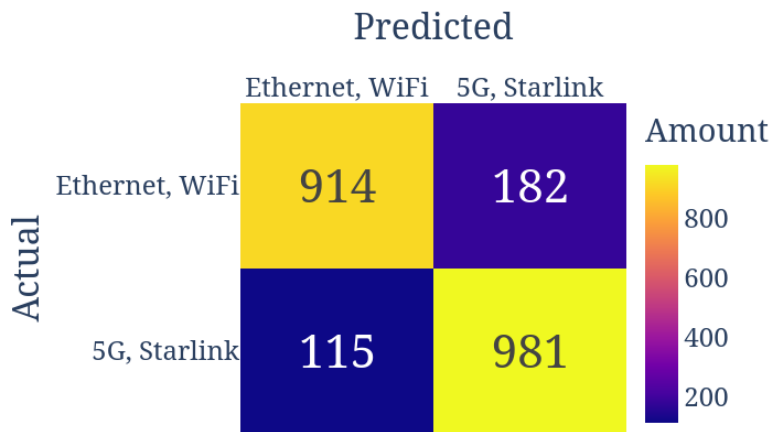


Figure 6.1.4: Sum of the confusion matrices from the Tor models *but* with Ethernet & WiFi as one group, and Starlink & 5G as another, 125 Epochs.

6.2 VPN — Multi-hop

As previously stated in Section 4.2, the parser used for VPN connections segments inter-arrival times into 1000-packet segments, since experiments showed this produced higher average accuracy, and this makes the training set sizes similar to that of Tor.

The average 10-fold accuracy as a function of training epochs for multi-hop VPN connections with pure inter-arrival times is shown in Figure 6.2.1. As seen in the figure, the average accuracy is similar to that of Tor (Section 6.1). However, we theorized that this accuracy could be limited by the possibility that the magnitudes of inter-arrival times were affected by the VPN server combinations used, and the resulting magnitudes were too distinct from one another. If the magnitudes are too distinct, the models might use the differences to identify the servers being used, making them

less generalizable because they might rely on that information. In Figure 6.2.2, the accuracy of two folds can be seen. The figure seems to indicate that the theory has some merit to it, since the two folds have significantly different maximum accuracies. To test if normalizing the inter-arrival times could help, Equation 6.1 was applied to the inter-arrival times of each sample. The algorithm implementing the Equation will start by applying the natural logarithm to each inter-arrival time in a segment, then calculate the mean and sample standard deviation of the logarithmized inter-arrival times. The new timestamp is then replaced with the logarithm of the inter-arrival time, minus the calculated mean, divided by the sample standard deviation. The new timestamp is also multiplied by the constant 3.6; this is an artifact from manual z-log score analysis that should not impact the results.

$$z = 3.6 \cdot \frac{\ln(t) - \overline{\ln(t)}}{s_{\ln(t)}}$$

Equation 6.1: Normalization for inter-arrival times, per segment.

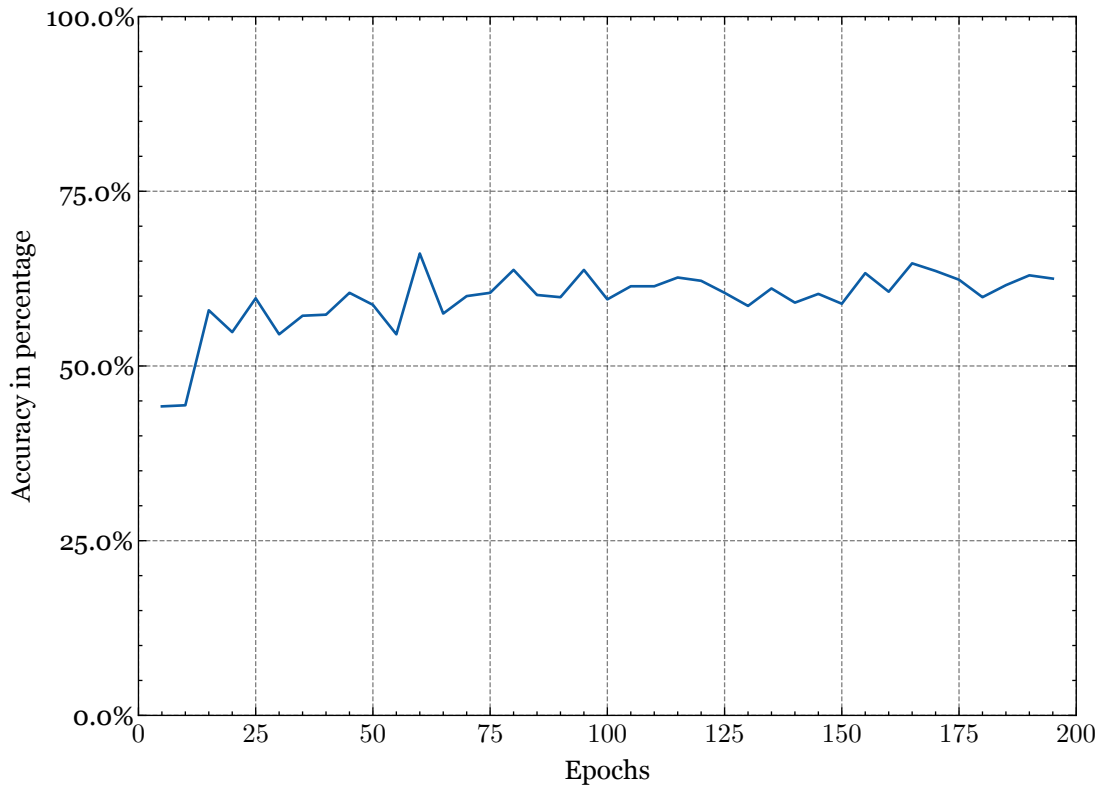


Figure 6.2.1: Average fold accuracy for VPN multi-hop for the different Epochs *before* normalization.

The average 10-fold accuracy as a function of training epochs for multi-hop VPN connections after applying the normalization algorithm (Equation 6.1) can be seen in Figure 6.2.3. The normalization had a positive effect, increasing the maximum accuracy from 66.1% to 87.8%.

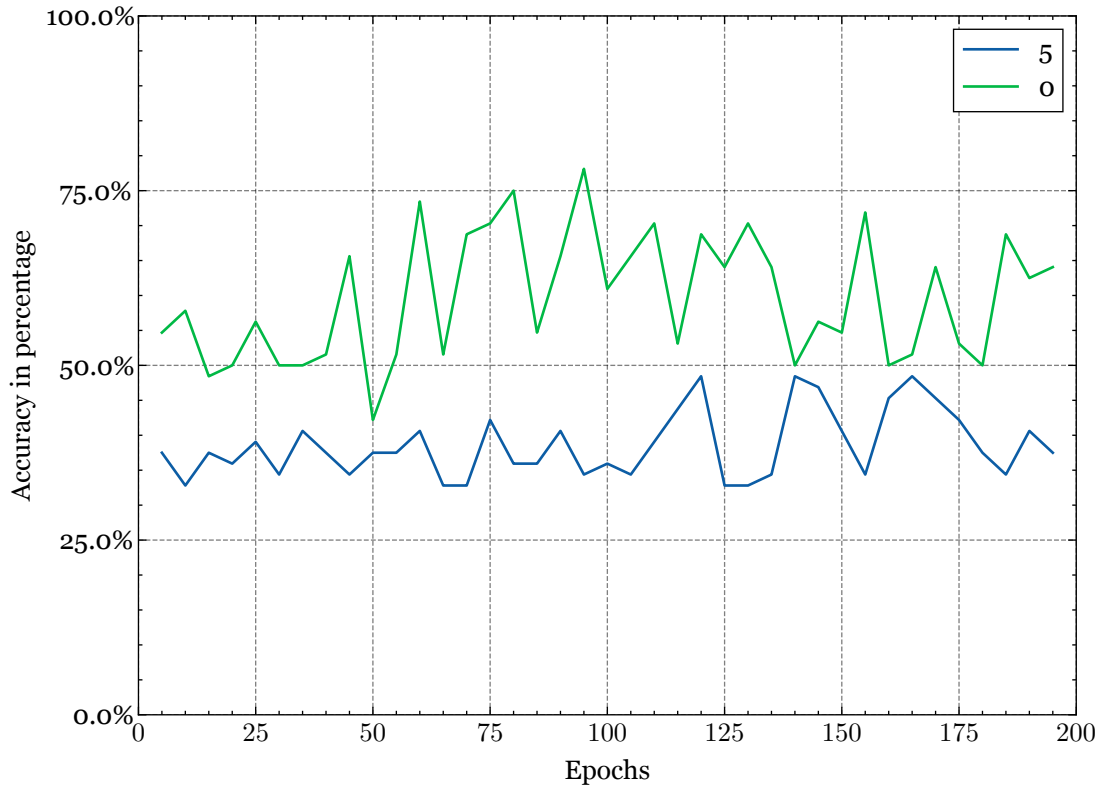


Figure 6.2.2: The fold performance of folds 0 and 5 for VPN multi-hop for the different Epochs *before* normalization.

As previously with Tor (Section 6.1), the average accuracy seems to fluctuate within a certain interval, 78.9-87.8%, after around 85 Epochs, indicating that more training will not yield significant benefits.

The accuracies for the two folds from Figure 6.2.2 can be seen in Figure 6.2.4. The maximum accuracy seems to be lower for certain folds and higher for others. This difference in accuracy *could* indicate that some VPN server pairs affect inter-arrival time patterns differently from others, meaning it affects not only the magnitude of inter-arrival times.

Just as with Tor, the number of samples (even after segmenting traces into more samples) is significantly lower than in the Tik-Tok paper (2368 vs 88500 samples in the Tik-Tok paper) [46], meaning a higher accuracy could potentially be achieved with more samples. The reason why the confusion matrix does not include the same number of samples as the total number of samples is that the Executor (Section 4.3) prevents a server from appearing in the training and validation set simultaneously, making the validation fold less than 10% of the total samples. The reported accuracy could be overestimated as well, since K-fold cross-validation has a tendency to overestimate the accuracy when using a small number of samples [49, 64, 65]. The small number of samples could also be an explanation for the high variance in accuracy for the two folds between epochs in Figure 6.2.4.

As seen in Table 6.2.1, the models trained with 150 epochs were, on average, most

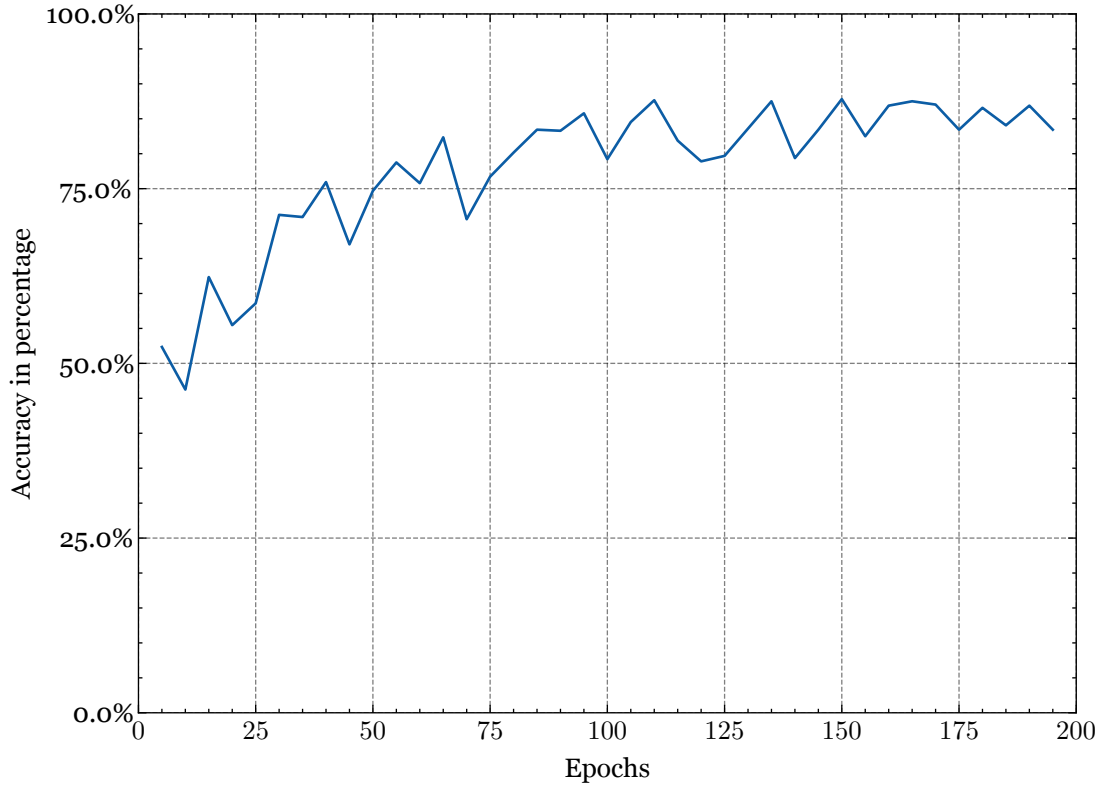


Figure 6.2.3: Average fold accuracy for VPN multi-hop for the different Epochs after applying normalization.

successful at detecting 5G connections with a recall value of 99.4%. Second most successful is once again Starlink with the recall 90.6%. Ethernet and WiFi accuracies have improved, with new recall values of 73.1% and 88.1%, respectively. The recall values for 5G and Starlink are similar to their respective precisions, indicating that the models are not too biased towards one of these connection types [43]. The precision of Ethernet is 17.1% higher than its recall, whilst the recall value of WiFi is 11.5% higher than its precision. This indicates that the models are biased toward selecting WiFi over Ethernet.

Table 6.2.1: Average CTFP recall of all folds for connection types behind a multi-hop VPN connection, 150 Epochs, normalized.

Connection type(s)	Recall (%)	Precision (%)
Ethernet	73.1	90.7
WiFi	88.1	76.6
5G	99.4	97.0
Starlink	90.6	89.0
Ethernet or WiFi (Same models)	93.4	95.5
5G or Starlink (Same models)	95.6	93.6

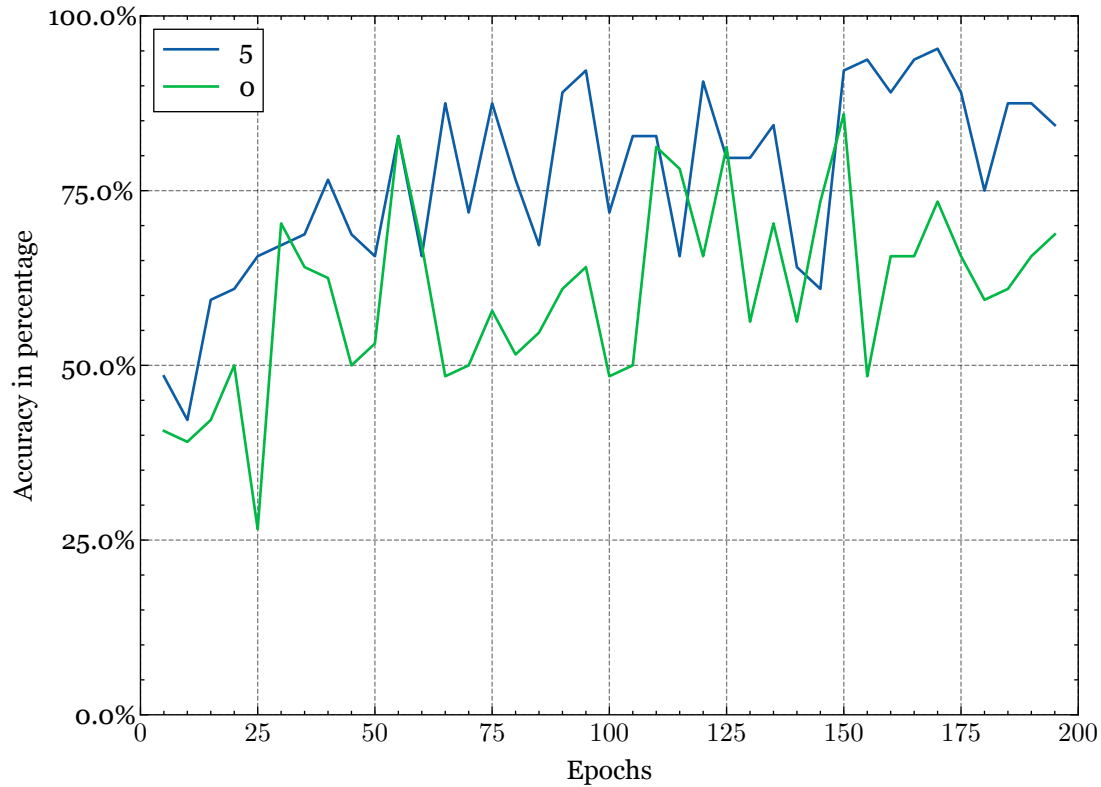


Figure 6.2.4: The fold performance of folds 0 and 5 for VPN multi-hop for the different Epochs after normalization.

The sum of the confusion matrices from models with 150 Epochs can be seen in Figure 6.2.5. As seen in the matrix, the model could distinguish between 5G and Starlink well and, to a lesser degree, between Ethernet and WiFi. A possible reason for the latter is that jitter between VPN servers, to some degree, obscures the inter-arrival patterns that distinguish Ethernet from WiFi.

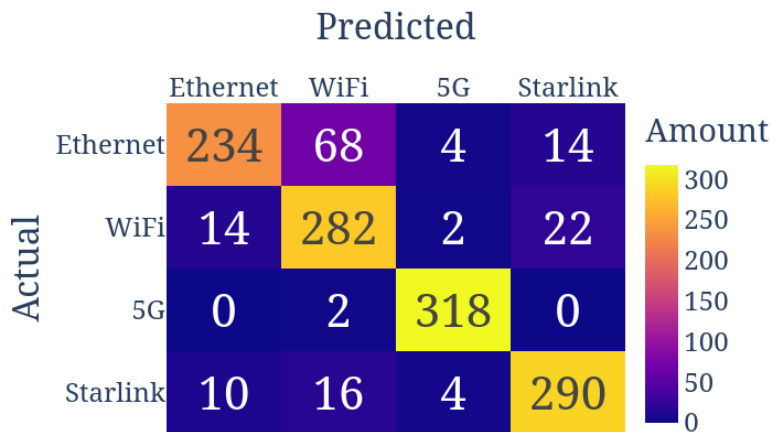


Figure 6.2.5: Sum of the confusion matrices from the VPN multi-hop models, 150 Epochs after normalization.

If we group together the classes Ethernet and WiFi from Figure 6.2.5 as one group, and the remaining two as a second group, we get a new confusion matrix as seen in Figure 6.2.6. The lower recall value for the Ethernet and WiFi group compared to the other group could be due to certain server combinations affecting inter-arrival times, making the inter-arrival time patterns similar to those of Starlink or 5G.

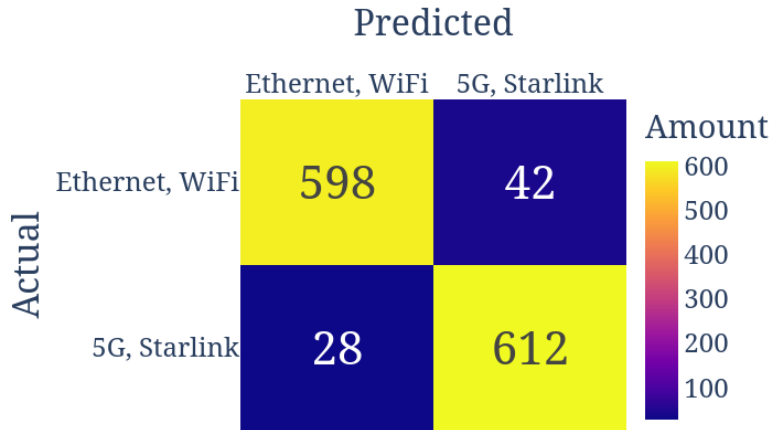


Figure 6.2.6: Sum of the confusion matrices from the VPN multi-hop models *but* with Ethernet & WiFi as one group, and Starlink & 5G as another, 150 Epochs, normalized.

6.3 VPN — One-hop

The one-hop models were trained with the parser that normalized the inter-arrival times (Section 6.2) since the magnitude of inter-arrival times could be different depending on the VPN server used.

The average 10-fold accuracy as a function of training epochs for one-hop VPN connections after applying the normalization algorithm (Equation 6.1) can be seen in Figure 6.3.1. As seen from the figure, the average accuracy reaches its maximum of 73.7% at 180 Epochs, 14.1% less than the maximum average accuracy of multi-hop connections. The aforementioned result might be due to the small sample size, 584 after segmentation, less than 25% of the multi-hop sample size.

The accuracies of folds 4 and 7 can be seen in Figure 6.3.2. Just as with multi-hop, some folds have higher maximum accuracies than others. Potentially, as a result of that, some servers may affect the inter-arrival patterns differently.

As seen in Table 6.3.1 and Figure 6.3.3, the models trained with 180 Epochs have decreased average performance compared to when multi-hop VPNs were used. The models for one-hop VPN connections still exhibit the biased behaviors between Ethernet and WiFi, as observed in the multi-hop models. The confusion matrix (Figure 6.3.4) shows that overall confusion has increased as well. All of these patterns *may* be a result of the smaller sample size compared to multi-hop.

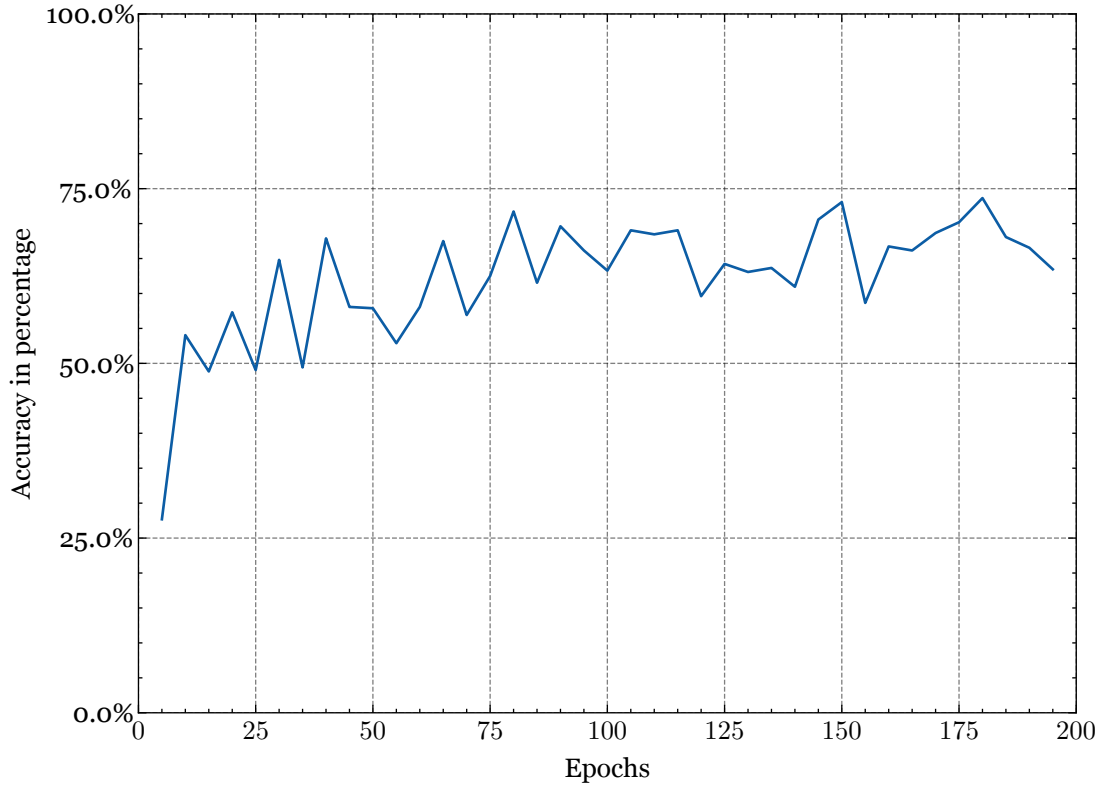


Figure 6.3.1: Average fold accuracy for VPN one-hop for the different Epochs. Timestamps normalized.

Table 6.3.1: Average CTFP recall of all folds for connection types behind a one-hop VPN connection, 180 Epochs, normalized.

Connection type(s)	Recall (%)	Precision (%)
Ethernet	45.4	62.8
WiFi	88.5	65.0
5G	87.7	87.0
Starlink	73.1	80.5
Ethernet or WiFi (Same models)	91.5	87.8
5G or Starlink (Same models)	87.3	91.2

Grouping the Ethernet and WiFi classes from Figure 6.3.3 into a single category, and the remaining two as a second category, we get a new confusion matrix as seen in Figure 6.3.4. From the figure, we can see that the models misclassified a 5G or Starlink connection type as WiFi or Ethernet 7.3% of the time, whilst the reverse was 19.6%, a difference of 12.3%.

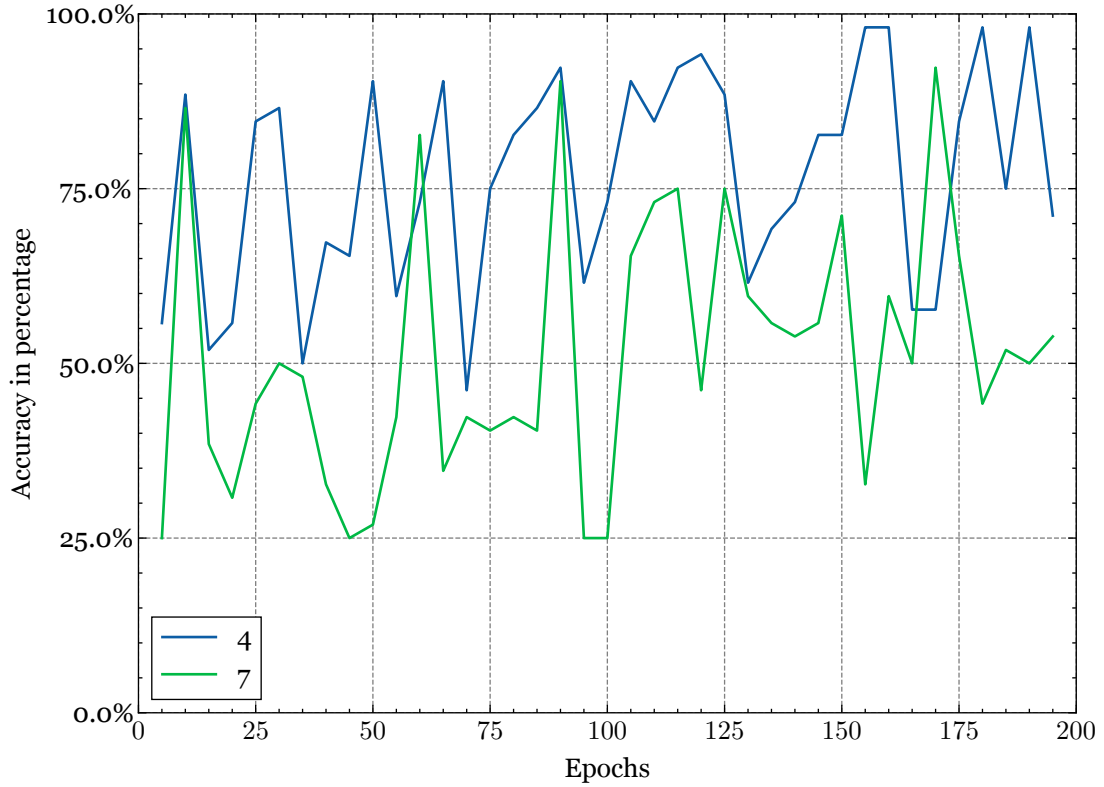


Figure 6.3.2: Each fold accuracy for VPN one-hop for the different Epochs. Timestamps normalized.

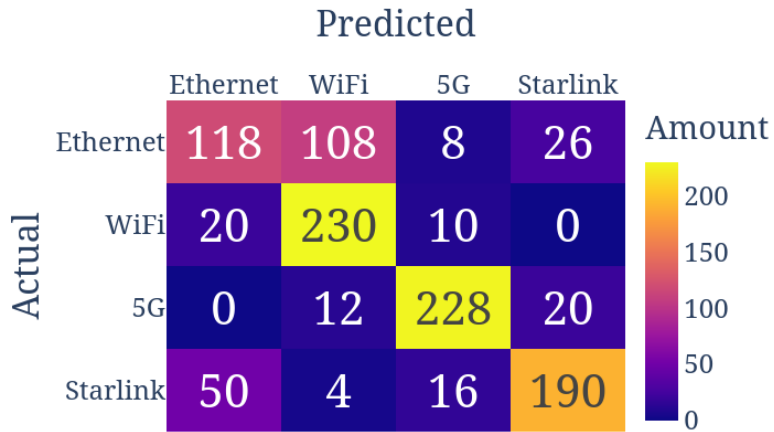


Figure 6.3.3: Sum of the confusion matrices from the VPN one-hop models, 180 Epochs, normalized.

6.4 No Relays

The performance of the Tik-Tok models when no relays were used can be seen in Figure 6.4.1. The inter-arrival times were normalized using Equation 6.1 to prevent the models from relying solely on latency for fingerprinting.

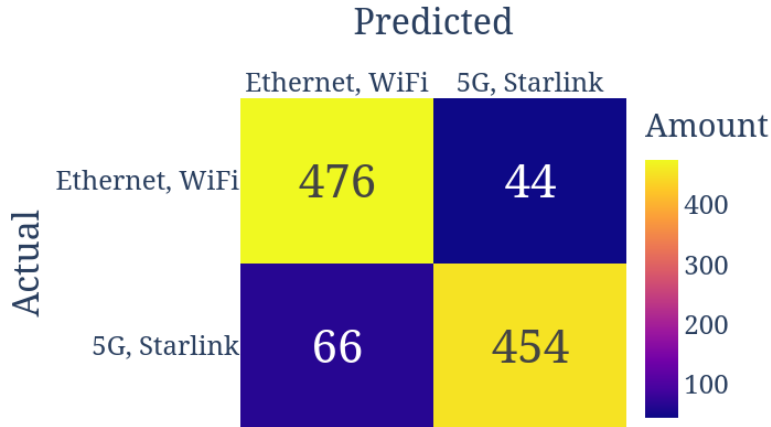


Figure 6.3.4: Sum of the confusion matrices from the VPN one-hop models *but* with Ethernet & WiFi as one group, and Starlink & 5G as another, 180 Epochs, normalized.

As expected (see Section 2.4), the models performed well, with an average of 99.7%, most likely because the characteristics of a connection type were not obfuscated by those of different relays. The total number of samples was 2240 *without* segmentation. This is further strengthened by the confusion matrix seen in Figure 6.4.2.

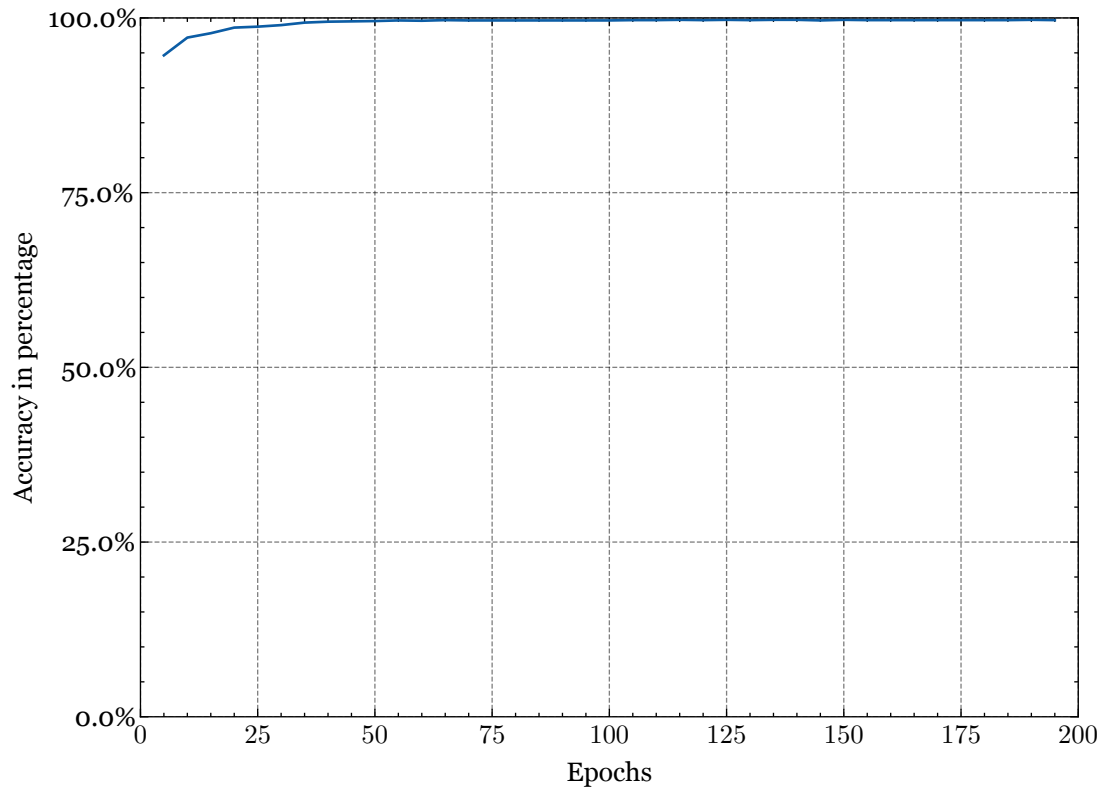


Figure 6.4.1: Average fold accuracy for when no relays are in use for the different Epochs. Normalized.

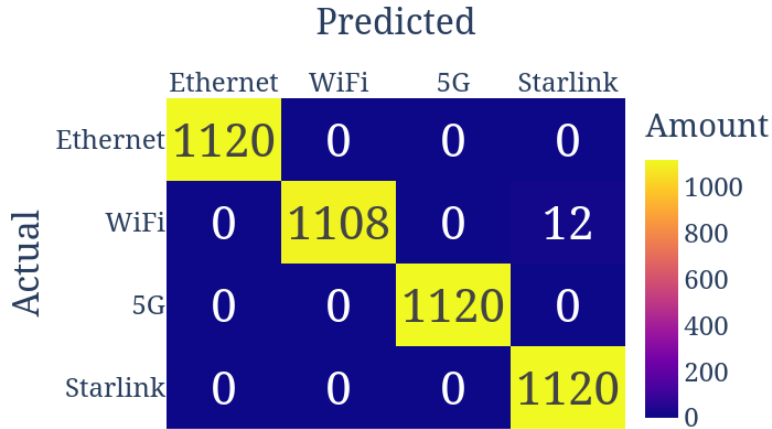


Figure 6.4.2: Sum of the confusion matrices for when no relays are in use, 190 Epochs, normalized.

6.5 Summary

A table summarizing the results discussed above can be seen in Table 6.5.1. In all scenarios, 5G and Starlink were detectable to a degree, whilst Ethernet and WiFi were not when 3 relays (Tor circuits) were used.

Table 6.5.1: Summary table of the results presented

Scenario	Best average accuracy	Best connection type recall	Worst connection type recall	Ethernet & WiFi seperable?
3 relays (Tor)	66.6%	5G (84.9%)	WiFi (45.4%)	No
2 relays (multi-hop VPN)	87.8%	5G (99.4%)	Ethernet (73.1%)	Yes
1 relay (one-hop VPN)	73.7%	WiFi (88.5%)	Ethernet (45.4%)	No (Ethernet misclassified as WiFi)
No relays	99.7%	Ethernet, 5G & Starlink (100.0%)	WiFi (98.9%)	Yes

As the table shows, the difficulty of CTFP appears to increase with the number of relays. However, since the preprocessing and the number of network traces differ between scenarios, a definitive answer cannot be drawn. Even if these two variables were the

same across scenarios, different techniques (for example, normalizing inter-arrival times) might be more appropriate in some scenarios than in others, making it non-trivial to determine which is more difficult and what to define as more difficult.

Chapter 7

Conclusions and Future Work

In this thesis, we investigated whether Connection-Type Fingerprinting (CTFP) is feasible for a classic client-server scenario in which the client is sending data to the server when using the Tik-Tok WF attack for CTFP. The investigation included the connection types Ethernet, WiFi, 5G, and Starlink. We also investigated how the performance of CTFP is affected while a client uses Tor circuits or single- or multi-hop VPNs when uploading to the server.

The results indicate that CTFP of Starlink and 5G was possible even when a client used three relays (Tor circuits). Separating Ethernet from WiFi was not achievable for Tor connections, but it was achievable with 2 or fewer relays. This finding indicates that the connection-type characteristics of 5G and Starlink affect Tor connection inter-arrival patterns sufficiently to be distinguishable from the others, whilst Ethernet and WiFi do not affect the inter-arrival patterns to such an extent that they become separable in the case of Tor. However, the small number of samples collected means that the accuracy achieved may be *overstated or understated*. Since the number of samples is so small, future work should include collecting more samples to see how this affects the results.

In the context of Tor and VPN services, having a server detect that a user is using 5G or Starlink could negatively impact the user's anonymity. The severity of it, however, depends on the number of other users of the service who use the same link technology. Anonymity systems hide one user among all other users of the service to provide anonymity. If the number of users who use 5G or Starlink is small, then the number of users one user effectively hides among becomes small as well.

However, as discussed throughout the thesis, several limitations have been identified that affect the results. First, only one VPN provider is used — Mullvad VPN. Using only one VPN provider is a significant limitation, as traffic patterns may vary across providers (see Section 2.3). Future work should attempt to perform CTFP with several VPN providers to assess how CTFP is affected by different VPN providers.

Another limitation is that we only used one network configuration for each connection

type, all of which were part of the Karlstad University laboratory network. Different network configurations may affect inter-arrival patterns of packets; for example, some WiFi connections may have stronger or weaker signals than others [42]. Also, we only considered four connection types, which does not include other common connection types used today, such as 4G and 3G. Therefore, the results of the thesis may not generalize to the real world, since different network configurations and connection types may affect inter-arrival patterns in distinct ways.

Yet another limitation is the use of VMs, and only two VM instances. The use of VMs affects how networking is performed on a machine; hence, it can affect the inter-arrival patterns of packets [48]. The network patterns may not only be affected by the use of VMs, but also directly by the hardware used. For example, there are NICs that can transfer data at 10 Gbit/s, whilst others may have higher or lower limits. Therefore, future work should examine how the results of the thesis would be affected by different hardware and by using physical machines instead of VMs.

We limited the bandwidth to 1 Mbit/s by using aggressive bandwidth control. This might, however, increase or decrease the distinguishability of different connection types. Future work should attempt to perform CTFP without bandwidth control, as in this experiment, and examine how different bandwidths affect the results of this thesis.

The scenario we considered was of clients uploading data to the server. Investigating other scenarios, such as file downloads or regular browsing, to determine how effective Tik-Tok is for CTFP is another avenue of future work.

Finally, the number of network configurations in this thesis was only four, one for each connection type, and all were part of the Karlstad University laboratory network. Future work should examine how different network configurations affect the results and determine whether CTFP remains possible under varied network configurations. Future work could also include different network conditions, since, for example, a mobile device's inter-arrival time patterns may be affected by weather or movement.

Bibliography

- [1] Bajpai, Vaibhav, Brunstrom, Anna, Feldmann, Anja, Kellerer, Wolfgang, Pras, Aiko, Schulzrinne, Henning, Smaragdakis, Georgios, Wählisch, Matthias, and Wehrle, Klaus. “The Dagstuhl beginners guide to reproducibility for experimental networking research”. In: *ACM SIGCOMM Computer Communication Review* 49.1 (2019), pp. 24–30.
- [2] Bartlett, Peter L, Long, Philip M, Lugosi, Gábor, and Tsigler, Alexander. “Benign overfitting in linear regression”. In: *Proceedings of the National Academy of Sciences* 117.48 (2020), pp. 30063–30070.
- [3] Bauer, Kevin, McCoy, Damon, Grunwald, Dirk, Kohno, Tadayoshi, and Sicker, Douglas. “Low-resource routing attacks against tor”. In: *Proceedings of the 2007 ACM Workshop on Privacy in Electronic Society*. WPES ’07. Alexandria, Virginia, USA: Association for Computing Machinery, 2007, pp. 11–20. ISBN: 9781595938831. DOI: 10.1145/1314333.1314336. URL: <https://doi.org/10.1145/1314333.1314336>.
- [4] Bhattacharjee, Arup and Nandi, Sukumar. “Statistical analysis of network traffic inter-arrival”. In: *2010 The 12th International Conference on Advanced Communication Technology (ICACT)*. Vol. 2. 2010, pp. 1052–1057.
- [5] Chen, Lijun, Low, Steven H., and Doyle, John C. “Random Access Game and Medium Access Control Design”. In: *IEEE/ACM Transactions on Networking* 18.4 (2010), pp. 1303–1316. DOI: 10.1109/TNET.2010.2041066.
- [6] Christensen, Kenneth J. “Performance evaluation of the binary logarithmic arbitration method (BLAM)”. In: *Proceedings of LCN-21st Annual Conference on Local Computer Networks*. IEEE. 1996, pp. 396–403.
- [7] Donenfeld, Jason A. “WireGuard: Next Generation Kernel Network Tunnel.” In: *NDSS*. 2017, pp. 1–12.
- [8] Dutkowska-Żuk, Agnieszka, Hounsel, Austin, Morrill, Amy, Xiong, Andre, Chetty, Marshini, and Feamster, Nick. “How and Why People Use Virtual Private Networks”. In: *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3451–3465. ISBN: 978-1-939133-31-1. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/dutkowska-zuk>.

- [9] Federrath, Hannes. *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability, Berkeley, CA, USA, July 25-26, 2000. Proceedings*. Springer, 2003.
- [10] Foong, A.P., Huff, T.R., Hum, H.H., Patwardhan, J.R., and Regnier, G.J. “TCP performance re-visited”. In: *2003 IEEE International Symposium on Performance Analysis of Systems and Software. ISPASS 2003*. 2003, pp. 70–79. DOI: 10.1109/ISPASS.2003.1190234.
- [11] Garcia, Johan, Sundberg, Simon, and Brunstrom, Anna. “A Detailed Characterization of Starlink One-way Delay”. In: *Proceedings of the 2025 3rd Workshop on LEO Networking and Communication*. LEO-NET '25. Coimbra, Portugal: Association for Computing Machinery, 2025, pp. 43–49. ISBN: 9798400720901. DOI: 10.1145/3748749.3749090. URL: <https://doi.org/10.1145/3748749.3749090>.
- [12] Gong, Jiajun, Cai, Wei, Liang, Siyuan, Guan, Zhong, Wang, Tao, and Chang, Ee-Chien. “Wfcat: Augmenting website fingerprinting with channel-wise attention on timing features”. In: *IEEE Transactions on Dependable and Secure Computing* (2025).
- [13] Goyal, Piyush and Goyal, Anurag. “Comparative study of two most popular packet sniffing tools-Tcpdump and Wireshark”. In: *2017 9th International Conference on Computational Intelligence and Communication Networks (CICN)*. 2017, pp. 77–81. DOI: 10.1109/CICN.2017.8319360.
- [14] Hinton, Geoffrey E, Srivastava, Nitish, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan R. “Improving neural networks by preventing co-adaptation of feature detectors”. In: *arXiv preprint arXiv:1207.0580* (2012).
- [15] Hogan, Kyle, Servan-Schreiber, Sacha, Newman, Zachary, Weintraub, Ben, Nita-Rotaru, Cristina, and Devadas, Srinivas. *ShorTor: Improving Tor Network Latency via Multi-hop Overlay Routing*. Apr. 2022. DOI: 10.48550/arXiv.2204.04489.
- [16] Huang, Guodong, Ma, Chuan, Ding, Ming, Qian, Yuwen, Ge, Chunpeng, Fang, Liming, and Liu, Zhe. “Efficient and low overhead website fingerprinting attacks and defenses based on TCP/IP traffic”. In: *Proceedings of the ACM Web Conference 2023*. 2023, pp. 1991–1999.
- [17] “IEEE Standard for Ethernet”. In: *IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018)* (2022), pp. 1–7025. DOI: 10.1109/IEEESTD.2022.9844436. URL: <https://ieeexplore.ieee.org/document/9844436>.
- [18] “IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems - Local and Metropolitan Area Networks–Specific Requirements - Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications - Corrigendum 1 – Correct IEEE 802.11ay Assignment of Protected Announce Support bit”. In: *IEEE Std 802.11-2020/Cor*

- 1-2022 (Corrigendum to IEEE Std 802.11-2020 as amended by IEEE Std 802.11ax-2021, IEEE Std 802.11ay-2021, and IEEE Std 802.11ba-2021)* (2022), pp. 1–18. DOI: 10.1109/IEEESTD.2022.9999411.
- [19] Kamal, Khaleque Md Aashiq and Almuhammadi, Sultan. “Vulnerability of virtual private networks to web fingerprinting attack”. In: *Advances in Security, Networks, and Internet of Things: Proceedings from SAM’20, ICWN’20, ICOMP’20, and ESCS’20*. Springer, 2021, pp. 147–165.
- [20] Kerrisk, Michael. *tc-police*. <https://man7.org/linux/man-pages/man8/tc-police.8.html>. Accessed: 2025-05-10.
- [21] Kim, Sungsoo, Yoo, Joon, and Choi, Jaehyuk. “Weigh-in-Motion: Lightweight Real-Time Identification of Gbps Wireless Traffic”. In: *Sensors* 22.2 (2022), p. 437.
- [22] Kunstmann, Anne, Bolonkowski, Gabriel, and See, August. “Enhancing Privacy in Tor: Message Mixing and Adaptive Padding with Opt-In Features”. In: *Proceedings of the 27th International Conference on Distributed Computing and Networking*. 2026, pp. 92–102.
- [23] Kweon, Seok-Kyu and Shin, K.G. “Statistical real-time communication over Ethernet”. In: *IEEE Transactions on Parallel and Distributed Systems* 14.3 (2003), pp. 322–335. DOI: 10.1109/TPDS.2003.1189588.
- [24] Landa, Raul, Clegg, Richard G., Araujo, Joao Taveira, Mykoniati, Eleni, Griffin, David, and Rio, Miguel. “Measuring the Relationships between Internet Geography and RTT”. In: *2013 22nd International Conference on Computer Communication and Networks (ICCCN)*. 2013, pp. 1–7. DOI: 10.1109/ICCCN.2013.6614151.
- [25] LeCun, Yann, Bengio, Yoshua, and Hinton, Geoffrey. “Deep learning”. In: *nature* 521.7553 (2015), pp. 436–444.
- [26] Lian, Jiayi, Freeman, Laura, Hong, Yili, and Deng, Xinwei. “Robustness with respect to class imbalance in artificial intelligence classification algorithms”. In: *Journal of Quality Technology* 53.5 (2021), pp. 505–525. DOI: 10.1080/00224065.2021.1963200. eprint: <https://doi.org/10.1080/00224065.2021.1963200>. URL: <https://doi.org/10.1080/00224065.2021.1963200>.
- [27] Liao, Weibin, Zhu, Yinghao, Wang, Xinyuan, Pan, Chengwei, Wang, Yasha, and Ma, Liantao. *LightM-UNet: Mamba Assists in Lightweight UNet for Medical Image Segmentation*. 2024. arXiv: 2403.05246 [eess.IV]. URL: <https://arxiv.org/abs/2403.05246>.
- [28] Lin, Xin, Lei, Lingguang, Wang, Yuewu, Jing, Jiwu, Sun, Kun, and Zhou, Quan. “A Measurement Study on Linux Container Security: Attacks and Countermeasures”. In: *Proceedings of the 34th Annual Computer Security Applications Conference. ACSAC ’18*. San Juan, PR, USA: Association for Computing Machinery, 2018, pp. 418–429. ISBN: 9781450365697. DOI: 10.1145/3274694.3274720. URL: <https://doi.org/10.1145/3274694.3274720>.

- [29] Loesing, Karsten, Murdoch, Steven J, and Dingledine, Roger. “A case study on measuring statistical data in the Tor anonymity network”. In: *International Conference on Financial Cryptography and Data Security*. Springer. 2010, pp. 203–215.
- [30] Magnaghi, Mattia, Ghezzi, Antonio, and Rangone, Andrea. “5G is not just another G: A review of the 5G business model and ecosystem challenges”. In: *Technological Forecasting and Social Change* 215 (2025), p. 124121.
- [31] Mathews, Nate, Holland, James K, Oh, Se Eun, Rahman, Mohammad Saidur, Hopper, Nicholas, and Wright, Matthew. “SoK: A Critical Evaluation of Efficient Website Fingerprinting Defenses”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. 2023, pp. 969–986. DOI: 10.1109/SP46215.2023.10179289.
- [32] Maxim Krasnyansky, Florian Thiel. *Universal TUN/TAP device driver*. <https://www.kernel.org/doc/html/latest/networking/tuntap.html>. Accessed: 2026-01-01. 2026.
- [33] Menon, Aravind and Zwaenepoel, Willy. “Optimizing TCP Receive Performance”. In: *2008 USENIX Annual Technical Conference (USENIX ATC 08)*. 2008.
- [34] Metcalfe, Robert M and Boggs, David R. “Ethernet: Distributed packet switching for local computer networks”. In: *Communications of the ACM* 19.7 (1976), pp. 395–404.
- [35] Michel, François, Trevisan, Martino, Giordano, Danilo, and Bonaventure, Olivier. “A first look at starlink performance”. In: *Proceedings of the 22nd ACM Internet Measurement Conference*. 2022, pp. 130–136.
- [36] Mohd Aminuddin, Mohamad Amar Irsyad, Zaaba, Zarul Fitri, Samsudin, Azman, Zaki, Faiz, and Anuar, Nor Badrul. “The rise of website fingerprinting on Tor: Analysis on techniques and assumptions”. In: *Journal of Network and Computer Applications* 212 (2023), p. 103582. ISSN: 1084-8045. DOI: <https://doi.org/10.1016/j.jnca.2023.103582>. URL: <https://www.sciencedirect.com/science/article/pii/S1084804523000012>.
- [37] Müller, Sebastian, Brecht, Franziska, Fabian, Benjamin, Kunz, Steffen, and Kunze, Dominik. “Distributed Performance Measurement and Usability Assessment of the Tor Anonymization Network”. In: *Future Internet* 4.2 (2012), pp. 488–513. ISSN: 1999-5903. DOI: 10.3390/fi4020488. URL: <https://www.mdpi.com/1999-5903/4/2/488>.
- [38] Mullvad. *Mullvad API Documentation*. <https://api.mullvad.net/accounts/v1/>. Accessed: 2026-01-01.
- [39] Mullvad. *Mullvad VPN Server List*. <https://api.mullvad.net/public/relays/wireguard/v1/>. The list of VPN servers available in json format, Accessed: 2026-01-01.
- [40] Mullvad. *Mullvad-VPN Client Source Code*. <https://github.com/mullvad/mullvadvpn-app>. Accessed: 2026-01-01.

- [41] Mullvad VPN. *Multihop with WireGuard*. Last updated: April 17, 2024. Mullvad VPN. 2024. URL: <https://mullvad.net/en/help/multihop-wireguard> (visited on 05/31/2026).
- [42] Murray, David, Koziniec, Terry, Dixon, Michael, and Lee, Kevin. “Measuring the reliability of 802.11 WiFi networks”. In: *2015 Internet Technologies and Applications (ITA)*. 2015, pp. 233–238. DOI: 10.1109/ITechA.2015.7317401.
- [43] Powers, David M. W. *Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation*. 2020. arXiv: 2010.16061 [cs.LG]. URL: <https://arxiv.org/abs/2010.16061>.
- [44] Project, The Tor. *Arti: reimplementing Tor in Rust*. <https://gitlab.torproject.org/tpo/core/arti#arti-reimplementing-tor-in-rust>. Accessed: 2026-01-01. 2026.
- [45] Pulls, Tobias and Witwer, Ethan. “Maybenot: A Framework for Traffic Analysis Defenses”. In: *Proceedings of the 22nd Workshop on Privacy in the Electronic Society*. WPES ’23. Copenhagen, Denmark: Association for Computing Machinery, 2023, pp. 75–89. ISBN: 9798400702358. DOI: 10.1145/3603216.3624953. URL: <https://doi.org/10.1145/3603216.3624953>.
- [46] Rahman, Mohammad Saidur, Sirinam, Payap, Mathews, Nate, Gangadhara, Kantha Girish, and Wright, Matthew. “Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks”. In: *CoRR abs/1902.06421* (2019). arXiv: 1902.06421. URL: <http://arxiv.org/abs/1902.06421>.
- [47] Rischke, Justus, Vielhaus, Christian, Sossalla, Peter, Itting, Sebastian, Nguyen, Giang T., and Fitzek, Frank H. P. “Empirical Study of 5G Downlink & Uplink Scheduling and its Effects on Latency”. In: *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. 2022, pp. 11–19. DOI: 10.1109/WoWMoM54355.2022.00017.
- [48] Rixner, Scot. “Network Virtualization: Breaking the Performance Barrier: Shared I/O in virtualization platforms has come a long way, but performance concerns remain.” In: *Queue* 6.1 (2008), 36–ff.
- [49] Shwartz-Ziv, Ravid, Goldblum, Micah, Bansal, Arpit, Bruss, C Bayan, LeCun, Yann, and Wilson, Andrew Gordon. “Just how flexible are neural networks in practice?” In: *arXiv preprint arXiv:2406.11463* (2024).
- [50] Sirinam, Payap, Imani, Mohsen, Juarez, Marc, and Wright, Matthew. “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning”. In: *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 2018, pp. 1928–1943.
- [51] Sommer, Jorg, Gunreben, Sebastian, Feller, Frank, Kohn, Martin, Mifdaoui, Ahlem, Saß, Detlef, and Scharf, Joachim. “Ethernet—a survey on its fields of application”. In: *IEEE Communications Surveys & Tutorials* 12.2 (2010), pp. 263–284.

- [52] SpaceX. *Starlink Progress Report*. Accessed: 2026-06-05. 2025. URL: <https://starlink.com/progress>.
- [53] Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. "Dropout: a simple way to prevent neural networks from overfitting". In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.
- [54] Sulaiman, Muhammad Aliyu and Zhioua, Sami. "Attacking tor through unpopular ports". In: *2013 IEEE 33rd International Conference on Distributed Computing Systems Workshops*. IEEE. 2013, pp. 33–38.
- [55] Syverson, P.F., Goldschlag, D.M., and Reed, M.G. "Anonymous connections and onion routing". In: *Proceedings. 1997 IEEE Symposium on Security and Privacy (Cat. No.97CB36097)*. 1997, pp. 44–54. DOI: 10.1109/SECPRI.1997.601314.
- [56] Sze, Vivienne, Chen, Yu-Hsin, Yang, Tien-Ju, and Emer, Joel S. "Efficient Processing of Deep Neural Networks: A Tutorial and Survey". In: *Proceedings of the IEEE* 105.12 (2017), pp. 2295–2329. DOI: 10.1109/JPROC.2017.2761740.
- [57] Tadros, Catherine Nayer, Rizk, Mohamed R. M., and Mokhtar, Bassem Mahmoud. "Software Defined Network-Based Management for Enhanced 5G Network Services". In: *IEEE Access* 8 (2020), pp. 53997–54008. DOI: 10.1109/ACCESS.2020.2980392.
- [58] TCPDump Group. *tcpdump: A Powerful Command-Line Packet Analyzer*. <https://www.tcpdump.org/manpages/tcpdump.1.html>. Accessed: 2026-01-01.
- [59] The kernel development community. *Segmentation Offloads*. <https://www.kernel.org/doc/html/latest/networking/segmentation-offloads.html>. Accessed: 2026-01-01.
- [60] The Tor Project. *OnionMasq*. <https://gitlab.torproject.org/tpo/core/onionmasq>. Accessed: 2026-01-01. 2026.
- [61] The Tor Project. *Research Safety Board*. <https://research.torproject.org/safetyboard/>. Accessed: 2026-05-31. 2024.
- [62] The Tor Project. *Tor Metrics Portal*. <https://metrics.torproject.org/>. Accessed: 2026-04-28. 2026.
- [63] The Tor Project. *Tor Path Selection Specification*. <https://spec.torproject.org/path-spec/index.html>. Accessed: 2026-04-28. 2026.
- [64] Vabalas, Andrius, Gowen, Emma, Poliakoff, Ellen, and Casson, Alexander J. "Machine learning algorithm validation with a limited sample size". In: *PLOS ONE* 14.11 (Nov. 2019), pp. 1–20. DOI: 10.1371/journal.pone.0224365. URL: <https://doi.org/10.1371/journal.pone.0224365>.

- [65] Varoquaux, Gaël. *Cross-validation failure: small sample sizes lead to large error bars*. 2017. arXiv: 1706.07581 [q-bio.QM]. URL: <https://arxiv.org/abs/1706.07581>.
- [66] Wei, Wei, Wang, Bing, Zhang, Chun, Kurose, Jim, and Towsley, Don. "Classification of Access Network Types: Ethernet, Wireless LAN, ADSL, Cable Modem or Dialup?" In: *Computer Networks* 52.17 (2008), pp. 3205–3217. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2008.08.018.
- [67] Wong, Tzu-Tsung and Yeh, Po-Yang. "Reliable Accuracy Estimates from k-Fold Cross Validation". In: *IEEE Transactions on Knowledge and Data Engineering* 32.8 (2020), pp. 1586–1594. DOI: 10.1109/TKDE.2019.2912815.
- [68] Wu, Zheng and Xiao, Mingzhong. "Performance Evaluation of VPN with Different Network Topologies". In: *2019 IEEE 2nd International Conference on Electronics Technology (ICET)*. 2019, pp. 51–55. DOI: 10.1109/ELTECH.2019.8839611.
- [69] Xiaofan, Wang, Chong, Peter HJ, and Yie, Leong Wai. "Performance comparison of csma/cd, csma/ca, csma/ri, csma/pri and csma/pr with beb". In: *2010 5th IEEE Conference on Industrial Electronics and Applications*. IEEE. 2010, pp. 1843–1848.
- [70] Xue, Diwen, Ramesh, Reethika, Jain, Arham, Kallitsis, Michaelis, Halderman, J Alex, Crandall, Jedidiah R, and Ensafi, Roya. "OpenVPN is open to VPN fingerprinting". In: *Communications of the ACM* 68.1 (2025), pp. 79–87.

Appendix - Contents

A Code Listings	50
B Tables	57

Appendix A

Code Listings

```
async fn create_device

1  async fn create_device(account : AsyncMullvadSlaveAccountEntry,
2    device_events : Arc<AsyncSharedQueue<DeviceEvents>>,
3    device_queue :
4      ↪ Arc<AsyncSharedQueue<(MullvadDevice, WireguardKeys)>>
5  ) -> Result<(), TrafficStarError>{
6    loop{
7      let keys = WireguardKeys::default();
8      let device =
9        ↪ account.inner.push_create_device_request(&keys).await?;
10     sdebug!("Created device {}", device.name);
11     let ipaddress = device.get_ipv4().unwrap();
12     let could_reserv = tokio::task::spawn_blocking(move || {
13       ReservationController::instance().reserv_ipaddress(&ipa_
14         ↪ ddress)
15     }).await.unwrap();
16     if !could_reserv{
17       error!("Mullvad device ip address {} given for {} is
18         ↪ already reserved!", &device.get_ipv4().unwrap(),
19         ↪ &device.name);
20       loop{
21         if let Err(err) =
22           ↪ account.inner.push_delete_device(&device).await{
23           error!("Failed deleting device {}, error :
24             ↪ {}", device.name, err);
25         }else{
26           sinfo!("Deleted device {} since it's address is
27             ↪ already reserved!", &device.name);
28           break;
29         }
30       }
31     }
32   }
```

```

21         }
22     }
23     continue;
24 }
25 account.add_known_device(device.clone(), WireguardKeys {
26     ↪ pubkey: keys.pubkey, privkey: keys.privkey }).await;
27 device_events.push(DeviceEvents::CreatedDevice);
28 device_queue.push((device,keys));
29 break;
30 }
31 Ok(())
32 }

```

Code Listing A.1: Part of the code used in the process to create Mullvad devices in TrafficStar

```

pub async fn use_peer

1 pub async fn use_peer(&self, peer : WireguardPeer, fwmark :
  ↪ Option<u32>) -> Result<(), TrafficStarError>{
2     let mut allowed_ips : Vec<IpNet> = Vec::new();
3     for allowed_ip in peer.allowedips.split(","){
4         allowed_ips.push(match IpNet::from_str(allowed_ip.trim()){
5             Ok(v) => {
6                 v},
7             Err(err) => {
8                 return Err(format!("Bad Allowed_Ips : {}, err :
  ↪ {}",&peer.allowedips, err).into())
9             },
10        });
11    }
12
13    let pubkey_peer = match
  ↪ wgctrl::types::Key::try_from(&peer.pubkey as &str){
14        Ok(v) => v,
15        Err(err) => {
16            return Err(format!("Unable to get pubkey, err
  ↪ {}",err).into())
17        },
18    };
19

```

```

20     let privkey = match
    ↪ wgctrl::types::Key::try_from(&*self.keys().privkey as
    ↪ &[u8]){
21         Ok(v) => v,
22         Err(err) => {
23             return Err(format!("Unable to get privkey, err
    ↪ {}",err).into())
24         },
25     };
26
27     let endpoint = match SocketAddr::from_str(&peer.endpoint){
28         Ok(v) => v,
29         Err(err) => {
30             return Err(format!("Unable to get endpoint from string
    ↪ {}, err {}",&peer.endpoint,err).into())
31         },
32     };
33
34
35     let config = wgctrl::types::Config {
36         private_key: Some(privkey),
37         listen_port: None,
38         firewall_mark: fwmark,
39         replace_peers: true,
40         peers: vec![PeerConfig{
41             public_key: pubkey_peer,
42             remove: false,
43             update_only: false,
44             preshared_key: None,
45             endpoint: Some(endpoint),
46             persistent_keepalive_interval:
    ↪ Some(Duration::from_secs(1)),
47             replace_allowed_ips: true,
48             allowed_ips },],
49     };
50
51
52     match wgctrl::client::Client::new(){
53         Ok(mut client) => {
54             if let Err(err) =
    ↪ client.configure_device(self.interface_name(),
    ↪ &config){

```

```

55         Err(format!("Error configuring device, err
           ↪ {}",err).into())
56     }else{
57         Ok(())
58     }
59 },
60 Err(err) =>
61 Err(format!("Error getting wg-client!, err {}",err).into()),
62 }
63
64
65 }

```

Code Listing A.2: Part of the code from TrafficStar used to create WireGuard interfaces

```

async fn run_sink, MullvadTestHandler

1  async fn run_sink(&self, sample : usize, output_directory :
   ↪ PathBuf, endhost : String, prefix : Option<String>)
2  -> Result<(),TrafficStarError>{
3
4      let (entry,exit,route) = &self.combinations_sink[sample %
   ↪ self.combinations_sink.len()];
5      let exit = exit.clone();
6      sinfo!("Entry : {}, exit : {}, interface : {}",entry.hostname,
   ↪ match exit.clone() {
7          Some(v) => {
8              v.hostname.clone()
9          }
10         None => {
11             "None".to_string()
12         }
13     }, route.interface_name);
14
15     let wg_pair = WgPeerPair{
16         entry : entry.clone(),
17         exit : exit.clone()
18     };
19
20     let mut route = route.clone();
21     let wgpeer = match entry.to_peer(exit.clone()){
22         Ok(v) => v,

```

```

23     Err(err) => return
    ↪ Err(TrafficStarError::msg(format!("Received error :
    ↪ {}",err)))
24 ,
25 };
26
27 route.info = Some(match serde_json::to_string(&wg_pair){
28     Ok(v) => v,
29     Err(err) => {
30         panicerror!("Couldn't create route info peer, error :
            ↪ {}",err)
31     },
32 });
33 sinfo!("Creating device!");
34 let vpn_connection = self.handler.get_device().await?;
35
36 vpn_connection.use_peer(wgpeer, route.fwmark).await?;
37
38 sinfo!("Created device {}!",vpn_connection.device().name);
39 route.interface_name =
    ↪ vpn_connection.interface_name().to_string();
40 route.ipv4 = vpn_connection.device().get_ipv4()?.to_string();
41
42
43 let result = TrafficStarTestSession::run_sink(self.config.sinkp_
    ↪ arams.clone().unwrap(), route, &endhost,
    ↪ output_directory,prefix).await;
44
45 sdebug!("Deleting VPN connection
    ↪ {}",vpn_connection.device().name);
46 vpn_connection.delete().await?;
47 sinfo!("Finished!");
48 result?;
49 Ok(())
50 }

```

Code Listing A.3: Code for running tests using VPN connections

```
async fn connect
```

```

1 async fn connect(&self, addr: &SocketAddr) ->
    ↪ IoResult<Self::Stream> {
2     trace!("New Arti TCP connection to {}", sensitive(addr));

```

```

3   let socket = if addr.is_ipv4() {
4       TcpSocket::new_v4()?
5   } else {
6       TcpSocket::new_v6()?
7   };
8
9   if let Err(err) = socket.bind_device(self.scaffolding.tor_bind_
    ↪ interface()).map(|v| &**v)){
10      return Err(err.into());
11  }
12  trace!("Tor connections using interface : {}", match
    ↪ self.scaffolding.tor_bind_interface(){
13      Some(v) =>
14      ↪ {String::from_utf8(v.clone()).unwrap_or("?".into())},
15      None => "None".into()
16  });
17
18  let fd = socket.as_raw_fd();
19  trace!("protecting fd {} for socket to {}", fd,
    ↪ sensitive(addr));
20  self.scaffolding.protect(fd, addr)?;
21  let inner = match socket.connect(*addr).await {
22      Ok(x) => {
23          trace!("Arti connection to {} successful",
24              ↪ sensitive(addr));
25          x
26      }
27      Err(e) => {
28          warn!("Arti failed to connect to {}: {}",
29              ↪ sensitive(addr), e);
30          return Err(e);
31      }
32  };
33  Ok(OnionTcpStream { inner })

```

Code Listing A.4: Code in the modified version of OnionMasq for creating a TCP connection to a Tor Guard relay

```

async fn run_sink, TorTestHandler

```

```

1  async fn run_sink(&self, sample : usize, output_directory :
    ↪ PathBuf, endpoint : String, prefix : Option<String>)

```

```

2 -> Result<(),TrafficStarError>{
3     sinfo!("Creating tor connection!");
4     let route = &self.combinations_sink[sample %
        ↪ self.combinations_sink.len()].clone();
5     let mut route = route.clone();
6
7     let interface_name = route.interface_name.clone();
8     let config_device = TorDeviceConfig::new(TrafficStarInterfaceNa
        ↪ me::from_str(&interface_name)?.await);
9     let handler = TorDevice::new(config_device).await?;
10
11     route.interface_name = handler.name().to_string();
12     route.ipv4 = handler.reservation.ipv4_addr.get_ip().to_string();
13     sinfo!("Tor device created, name : {}, binding interface :
        ↪ {}",route.interface_name, interface_name);
14     let result = TrafficStarTestSession::run_sink(
        ↪ self.config.sinkparams.clone().unwrap(),route, &endhost,
        ↪ output_directory,prefix).await;
15
16     sdebug!("Finished!");
17     handler.stop().await?;
18     sdebug!("Killed handler!");
19     result?;
20     Ok(())
21 }

```

Code Listing A.5: Code for running tests using Tor connections

Appendix B

Tables

Table B.0.1: Number of network traces collected containing a VPN server for multi-hop VPN connections.

Connection type	Country	City	Server	Count
Ethernet	Argentina	Buenos Aires	ar-bue-wg-002	76
	Belgium	Brussels	be-bru-wg-101	72
	Bulgaria	Sofia	bg-sof-wg-001	63
	Canada	Montreal	ca-mtr-wg-304	63
	France	Bordeaux	fr-bod-wg-002	63
	Ireland	Dublin	ie-dub-wg-103	63
	Japan	Osaka	jp-osa-wg-004	62
	Norway	Oslo	no-osl-wg-102	62
	Spain	Madrid	es-mad-wg-102	62
	Sweden	Göteborg	se-got-wg-003	62
	USA	Phoenix, AZ	us-phx-wg-208	62
5G	Argentina	Buenos Aires	ar-bue-wg-002	100
	Belgium	Brussels	be-bru-wg-101	99
	Bulgaria	Sofia	bg-sof-wg-001	98
	Canada	Montreal	ca-mtr-wg-304	98
	France	Bordeaux	fr-bod-wg-002	99
	Ireland	Dublin	ie-dub-wg-103	98
	Japan	Osaka	jp-osa-wg-004	97
	Norway	Oslo	no-osl-wg-102	98
	Spain	Madrid	es-mad-wg-102	99
	Sweden	Göteborg	se-got-wg-003	98
	USA	Phoenix, AZ	us-phx-wg-208	90
	Argentina	Buenos Aires	ar-bue-wg-002	58

Continued on next page

	Belgium	Brussels	be-bru-wg-101	58
	Bulgaria	Sofia	bg-sof-wg-001	57
	Canada	Montreal	ca-mtr-wg-304	57
	France	Bordeaux	fr-bod-wg-002	57
	Ireland	Dublin	ie-dub-wg-103	57
	Japan	Osaka	jp-osa-wg-004	57
	Norway	Oslo	no-osl-wg-102	57
	Spain	Madrid	es-mad-wg-102	48
	Sweden	Göteborg	se-got-wg-003	48
	USA	Phoenix, AZ	us-phx-wg-208	48
WiFi	Argentina	Buenos Aires	ar-bue-wg-002	105
	Belgium	Brussels	be-bru-wg-101	102
	Bulgaria	Sofia	bg-sof-wg-001	92
	Canada	Montreal	ca-mtr-wg-304	95
	France	Bordeaux	fr-bod-wg-002	86
	Ireland	Dublin	ie-dub-wg-103	84
	Japan	Osaka	jp-osa-wg-004	85
	Norway	Oslo	no-osl-wg-102	85
	Spain	Madrid	es-mad-wg-102	84
	Sweden	Göteborg	se-got-wg-003	85
	USA	Phoenix, AZ	us-phx-wg-208	81

Table B.O.2: Number of network traces collected per VPN server for one-hop VPN connections.

Connection type	Country	City	Server	Count
Ethernet (102)	Argentina	Buenos Aires	ar-bue-wg-002	20
	Belgium	Brussels	be-bru-wg-101	20
	Bulgaria	Sofia	bg-sof-wg-001	20
	Canada	Montreal	ca-mtr-wg-304	20
	France	Bordeaux	fr-bod-wg-002	18
	Ireland	Dublin	ie-dub-wg-103	18
	Japan	Osaka	jp-osa-wg-004	18
	Norway	Oslo	no-osl-wg-102	18
	Spain	Madrid	es-mad-wg-102	18
	Sweden	Göteborg	se-got-wg-003	18
	USA	Phoenix, AZ	us-phx-wg-208	16
	Argentina	Buenos Aires	ar-bue-wg-002	12
	Belgium	Brussels	be-bru-wg-101	12

Continued on next page

	Bulgaria	Sofia	bg-sof-wg-001	12
	Canada	Montreal	ca-mtr-wg-304	10
	France	Bordeaux	fr-bod-wg-002	10
	Ireland	Dublin	ie-dub-wg-103	10
	Japan	Osaka	jp-osa-wg-004	10
	Norway	Oslo	no-osl-wg-102	10
	Spain	Madrid	es-mad-wg-102	10
	Sweden	Göteborg	se-got-wg-003	10
	USA	Phoenix, AZ	us-phx-wg-208	10
Starlink (161)	Argentina	Buenos Aires	ar-bue-wg-002	34
	Belgium	Brussels	be-bru-wg-101	32
	Bulgaria	Sofia	bg-sof-wg-001	26
	Canada	Montreal	ca-mtr-wg-304	30
	France	Bordeaux	fr-bod-wg-002	30
	Ireland	Dublin	ie-dub-wg-103	30
	Japan	Osaka	jp-osa-wg-004	30
	Norway	Oslo	no-osl-wg-102	26
	Spain	Madrid	es-mad-wg-102	28
	Sweden	Göteborg	se-got-wg-003	28
	USA	Phoenix, AZ	us-phx-wg-208	28
WiFi (68)	Argentina	Buenos Aires	ar-bue-wg-002	18
	Belgium	Brussels	be-bru-wg-101	12
	Bulgaria	Sofia	bg-sof-wg-001	12
	Canada	Montreal	ca-mtr-wg-304	12
	France	Bordeaux	fr-bod-wg-002	12
	Ireland	Dublin	ie-dub-wg-103	12
	Japan	Osaka	jp-osa-wg-004	10
	Norway	Oslo	no-osl-wg-102	12
	Spain	Madrid	es-mad-wg-102	12
	Sweden	Göteborg	se-got-wg-003	12
	USA	Phoenix, AZ	us-phx-wg-208	12